

Arquitectura de las computadoras:

Resumen examen final 2015.

J.G

FRC UTN 1k10

Unidad I: Sistemas numéricos.

SISTEMAS NUMERICOS DE DISTINTAS BASES:

Sistemas de notación posicional:

Los sistemas de notación posicional están formados por una n cantidad de símbolos, cuya combinación representa valores diferentes. Cada dígito tiene un peso diferente según el lugar que ocupa. El peso es la base elevada a la posición que ocupa dentro del número. La suma del peso de cada dígito permitirá obtener el valor final.

En todo sistema posicional de base B , se obtienen B dígitos posibles: de 0 a $B-1$.

Expresión generalizada de un número en potencias de su base:

Dado un número n en base B expresado por n dígitos, este será igual a la sumatoria de cada dígito por la potencia de la base B que le corresponde a su posición:

Sistema decimal:

Es un sistema de notación posicional formado por 10 símbolos (0,1,2,3,4,5,6,7,8,9)

Sistema binario:

Es un sistema de notación posicional formado por dos símbolos (0,1) a los que se denomina bits. Un número binario está compuesto por un conjunto de bits y su equivalente decimal será la suma que resulte de multiplicar a cada bit por su peso según la posición que ocupen dentro del número. Este sistema es el usado en las computadoras digitales.

Conveniencia tecnológica del sistema binario:

El sistema binario ofrece muchas ventajas en el manejo interno de la información en las computadoras. Esto se debe a la mayor simplicidad de diseño de una unidad aritmética binaria o a la facilidad de transferir y almacenar magnitudes físicas que representen bits.

Operar tecnológicamente con dos estados es mucho más simple y confiable que hacer trabajar a los circuitos con (por ejemplo) 10 estados.

Agrupaciones de bits:

Un solo dígito binario se llama bit, cuatro bits agrupados se denominan nibble. Ocho bits agrupados se denominan byte.

Tamaño de palabra:

Una palabra es un grupo de bits que es procesada como un simple número o instrucción por el microprocesador. Las palabras más comunes son de 4, 8, 16 o 32 bits.

Sistema octal:

Es un sistema de notación posicional formado por 8 símbolos (0, 1, 2, 3, 4, 5, 6, 7)

Sistema hexadecimal:

Es un sistema de notación posicional formado por 16 símbolos (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F) que representan los valores decimales del 0 al 15.

La mayoría de los sistemas de microcomputadoras utilizan la notación hexadecimal para simplificar la tarea de recordar y teclear números binarios.

El 10(uno-cero) es el número que en cualquier sistema representa la base
--

Para analizar los contenidos de los registros internos de la computadora, se utilizan los sistemas octal y hexadecimal ya que permiten compactar bits en grupos de 3 o 4.

Métodos de conversión de números de otras bases a decimal

Binario a decimal

- Enteros: El valor decimal es igual a multiplicar cada bit por la potencia que corresponda a su posición y sumar los valores finales.
- Fracciones: Se obtiene el valor decimal multiplicando cada bit por la potencia negativa de dos que corresponde a su posición a partir de la coma, y sumando los valores finales.

Octal a decimal

- Se obtendrá el valor decimal multiplicando cada dígito octal por la potencia de ocho que corresponda a la posición y sumando los valores finales

Hexadecimal a decimal:

- Se obtendrá el valor decimal multiplicando cada dígito hexadecimal por la potencia de dieciséis que corresponda a la posición y sumando los valores finales.

Conversión de un número entero decimal a otras bases

Se divide el número por la base que corresponde hasta hallar el último cociente. Se formará con todos los restos anteriores (desde el último al primero) el número buscado.

Conversión de un número fraccionario decimal a otras bases:

Cuando un número tiene parte entera y fraccionaria, se lo separa y se aplica un método para la conversión de la parte entera y otro para la conversión de la parte decimal.

1. Multiplicar la fracción por la base de conversión, cuyo resultado tiene una parte entera que se considerará uno de los dígitos fraccionarios buscados.
2. Del resultado anterior se toma solo la parte fraccionaria y se vuelve a multiplicar por la base. Se repite este paso tantas veces hasta cancelar el método. El método se cancela cuando:
 - a. La parte fraccionaria se hace 0
 - b. Se repite una secuencia de números fraccionarios, es una fracción periódica

El nuevo número fraccionario estará compuesto por la parte entera resultante de cada multiplicación desde la primera hasta la última. Deben tomarse en cuenta las siguientes consideraciones:

- Cada parte entera obtenida no debe superar el valor de la base del sistema
- Cada parte entera debe representarse con un solo símbolo del sistema al que se convierte.

Pasaje directo entre las bases 2 a 8 y 2 a 16

Este pasaje puede realizarse porque 8 y 16 son potencias exactas de 2.

- Binario a octal:
 1. Se divide el número binario en grupos de 3 bits. Para enteros desde el bit de menor peso al de mayor peso y para fracciones en sentido inverso
 2. Se sustituye cada grupo de bits por el correspondiente dígito octal.
- Octal a binario: Se reemplaza cada dígito octal por el equivalente binario en grupo de 3 bits.
- Binario a hexadecimal: Se repite el proceso para convertir de binario a octal, solo que se agrupan 4 bits en lugar de 3
- Hexadecimal a binario: Se reemplaza cada dígito hexadecimal por el equivalente en binario en grupos de 4 bits.

Operaciones fundamentales en binario

Suma

La suma entre 2 bits tiene 4 combinaciones posibles:

1. $0+0=0$
2. $0+1=1$
3. $1+0=1$

4. $1+1=0$ y acarreo 1

Resta

1. $0-0=0$
2. $0-1=1$ Le pide 1 a la posición siguiente(acarreo)
3. $1-0=1$
4. $1-1=0$

Resta por complemento

Permite restar mediante la suma

- Complemento a la base: En todo sistema numérico de notación posicional, para un número x de n dígitos existe un número y de n dígitos que es su complemento a la base (o lo que le falta para llegar a la base) : $x + y = B$ o $C_b N = B^n - N$ (siendo 10 la base de cualquier sistema) En el sistema binario el complemento de un número es su inverso + 1
- Complemento a la base -1: El complemento a la base menos uno de un número es el resultado de elevar la base a la potencia dada por la cantidad de cifras del número, menos 1 y luego restarle el número dado. $C_{b-1} N = B^{n-1} - N$. En el sistema binario para hallar el complemento a la base menos uno se cambian los 0 por 1 y al revés.

Flujo de datos dentro de una computadora:

Tanto las instrucciones de un programa como los datos que ingresan a la memoria desde el exterior lo hacen en un código alfanumérico de representación de caracteres.

Un código alfanumérico establece la relación necesaria para que una computadora digital interprete el lenguaje que utiliza el usuario.

Un código establece una combinación binaria para cada símbolo que se ha de representar.

Códigos de representación de caracteres alfanuméricos

- Código ASCII: Este código de 7 bits representa cada carácter con 7 bits. Lo que le permite determinar $2^7 = 128$ combinaciones diferentes. Los caracteres que pueden representarse a través de este código son: letras mayúsculas y minúsculas, números del 0 al 9, signos de puntuación y caracteres de control.
- ASCII de 8 bits: El uso intensivo del código ASCII de 7 bits dio lugar a la necesidad de agregar caracteres. Con 8 bit se determinan $2^8 = 256$ combinaciones diferentes. Las primeras 128 combinaciones determinan los caracteres originales y las 128 combinaciones siguientes representan caracteres gráficos y no esenciales.

Códigos de representación decimal (BCD)

Son convenciones que permiten la representación de números decimales (0 a 9) en bloques binarios de 4 bits. Se los denomina ponderados porque adjudican peso a los 1 binarios, según la posición que ocupan en el bloque. La suma de los pesos de cada combinación será igual al número decimal representado.

Hay 3 clases de códigos BCD: El BCD puro y dos derivaciones de él.

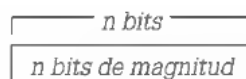
- BCD puro o natural: Los pesos en cada bloque coinciden con el valor de los 4 primeros pesos del sistema binario puro, es decir 8421
- BCD exceso en 3: Se obtiene a partir del BCD puro sumando un 3 binario a cada cifra decimal (es decir arranca en 0011). Es un código simétrico o autocomplementado, es decir facilita la operación de hallar el complemento de un número solo con invertir sus dígitos
- BCD Aiken: El peso de este código para los 1 binarios es 2421. Es un código simétrico ya que permite hallar el complemento de un número invirtiendo los bits de la combinación.

Expresión generalizada de representaciones de cantidades enteras y fracciones

Estos códigos permiten la operación aritmética de datos en sistema binario u otras bases no decimales. Los operandos ingresan en código alfanumérico para luego convertirse a alguno de estos convenios.

Coma o punto fijo sin signo (números enteros)

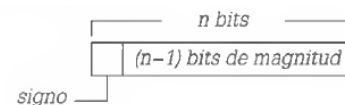
Formato más simple para representar números enteros positivos, expresados como una secuencia de binarios. Para la representación de un número se utiliza la totalidad de bits del formato.



Permite representar los números comprendidos en ese rango. Si la suma de dos números supera el rango de variabilidad estipulado, el cálculo es erróneo ya que perderá el bit más significativo. Existen señalizadores (flags) que indican circunstancias de desborde u “overflow”.

Coma o punto fijo con signo

Es igual al formato anterior pero reserva el bit de extrema izquierda para el signo. 0 indicará que el número es positivo, mientras que un 1 indicará lo contrario.



El rango de variabilidad para los binarios con signo es: $-(2^{n-1} - 1) \leq x \leq (2^{n-1} - 1)$

Coma o punto fijo con signo con negativos complementados a 2

Al igual que el formato anterior, reserva el bit de extrema izquierda al signo. los n-1 bits restantes se representan con su magnitud binaria real para los positivos y en el complemento a la base de la magnitud en el caso de los negativos.

Coma o punto fijo con signo con negativos complementados a 1

Al igual que los dos formatos anteriores, reserva el bit de extrema izquierda al signo. Los n-1 bits restantes se representan con su magnitud binaria real para los positivos y en el complemento a 1 de la magnitud en el caso de los negativos.

Reales en coma o punto flotante (números muy grandes y números reales)

Para representar valores muy grandes se amplía el formato a 3, 4 u 8 octetos.

Para representar el punto a la hora de manejar cantidades fraccionarias, el mismo no asume una posición fija dentro de la cadena. Es así que los números se representan considerando dos partes:

1. La primera indica los bits que representan el número y se llama ***mantisa***.
2. La otra indica la posición del punto dentro del número y se llama ***exponente***.

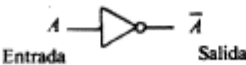
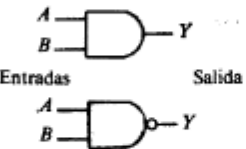
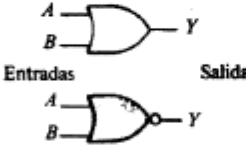
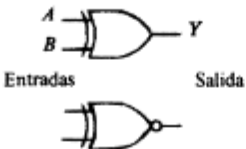
Unidad II: Circuitos lógicos. Algebra de los circuitos digitales.

Algebra de los circuitos digitales.

Compuertas lógicas

Para cada combinación de entradas la función asume un valor 0 o 1.

Funciones lógicas

Función lógica	Símbolo lógico de puerta	Tabla de verdad	Expresión booleana																								
Inversor	 Entrada: A, Salida: $\bar{A}$	<table><tr><th>Entrada</th><th>Salida</th></tr><tr><th>A</th><th>$\bar{A}$</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	Entrada	Salida	A	$\bar{A}$	0	1	1	0	$A = \bar{A}$																
Entrada	Salida																										
A	$\bar{A}$																										
0	1																										
1	0																										
AND NAND	 Entradas: A, B; Salidas: Y	<table><tr><th colspan="2">Entradas</th><th colspan="2">Salidas</th></tr><tr><th>B</th><th>A</th><th>AND</th><th>NAND</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	Entradas		Salidas		B	A	AND	NAND	0	0	0	1	0	1	0	1	1	0	0	1	1	1	1	0	$A \cdot B = Y$ $\overline{A \cdot B} = Y$
Entradas		Salidas																									
B	A	AND	NAND																								
0	0	0	1																								
0	1	0	1																								
1	0	0	1																								
1	1	1	0																								
OR NOR	 Entradas: A, B; Salidas: Y	<table><tr><th colspan="2">Entradas</th><th colspan="2">Salidas</th></tr><tr><th>B</th><th>A</th><th>OR</th><th>NOR</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	Entradas		Salidas		B	A	OR	NOR	0	0	0	1	0	1	1	0	1	0	1	0	1	1	1	0	$A + B = Y$ $\overline{A + B} = Y$
Entradas		Salidas																									
B	A	OR	NOR																								
0	0	0	1																								
0	1	1	0																								
1	0	1	0																								
1	1	1	0																								
OR exclusiva NOR exclusiva	 Entradas: A, B; Salidas: Y	<table><tr><th colspan="2">Entradas</th><th colspan="2">Salidas</th></tr><tr><th>B</th><th>A</th><th>XOR</th><th>XNOR</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td><td>1</td></tr></table>	Entradas		Salidas		B	A	XOR	XNOR	0	0	0	1	0	1	1	0	1	0	1	0	1	1	0	1	$A \oplus B = Y$ $\overline{A \oplus B} = Y$
Entradas		Salidas																									
B	A	XOR	XNOR																								
0	0	0	1																								
0	1	1	0																								
1	0	1	0																								
1	1	0	1																								

Combinación de puertas lógicas

Los sistemas digitales están formados por combinaciones de puertas lógicas, las cuales pueden ser descritas por una tabla de verdad, expresiones booleanas o diagramas lógicos.

Tabla de verdad

Lista ordenada de las 2^n combinaciones distintas de ceros y unos, que se pueden obtener de la combinación del valor de n variables binarias. Con n variables se pueden obtener 2^n combinaciones distintas.

Función canónica de una función

Una función se puede representar por una serie de términos canónicos. Un **término canónico** es todo producto o suma en los que aparecen todas las variables que componen una función, en su forma directa o inversa.

- minitérmino: Es el producto de las variables en juego.
- maxitérmino: Es la suma de las variables en juego.

Observando la tabla de verdad se elegirá representar las funciones de la forma que más nos convenga, teniendo en cuenta la menor cantidad de términos que se han de representar.

Simplificación de funciones

La minimización se realiza para que el diseño de un circuito sea lo más simple posible, o sea, cumpla la misma función con la mínima cantidad de compuertas posibles.

Mapa de Karnaugh

Es un esquema gráfico con 2^n casillas, siendo n las entradas del circuito. Donde se utiliza el código de Gray para representar la simplificación de una expresión o función lógica. Para expresar Karnaugh se necesita la expresión canónica para poder situar las combinaciones en las casillas y simplificar.

Código de Gray: Código cuya característica es que entre combinaciones lógicamente adyacentes haya solo una variación de bit.

La simplificación se realiza agrupando la mayor cantidad de 1 adyacentes posibles, siempre y cuando esta cantidad sea una potencia exacta de 2. Luego, la función mínima será la unión de todos los grupos definidos antes.

Algebra de Boole

Es un ente matemático constituido por los elementos 0 y 1 y las operaciones lógicas suma, producto y complemento.

Propiedades del algebra de Boole

- Leyes conmutativas: Ambas operaciones binarias son conmutativas: $a+b = b+a$ / $a.c = c.a$
- Leyes distributivas: Cada operación binaria es distributiva respecto de la otra: $a.(b+c) = (a.b)+(a.c)$
- Leyes de identidad: Existen dos elementos neutros, el 0 y el 1 respecto a cada una de las operaciones. $a+0 = a$ / $a.1=a$
- Leyes de complementación: Para cada elemento a del algebra, existe un elemento denominado a negada. $a + \sim a = 1$ / $a.\sim a=0$

Teoremas del algebra de Boole

1. Principio de dualidad: Las expresiones permanecen válidas si se intercambian las operaciones + por . y los elementos 0 y 1.
2. Teorema 2: Para cada elemento del Algebra de Boole se verifica que: $a+1 = 1$ / $a.0=0$
3. Teorema 3: Para cada elemento del Algebra de Boole se verifica que: $a+a= a$ / $a.a=a$
4. Teorema 4: Para cada par de elementos del algebra de Boole se verifica que: $a+(a.b)= a$ / $a.(a+b)=a$
5. Teorema 5: Suma y producto son asociativos: $a+b+c = (a+b) +c = a+(b+c)$ / $a.b.c = (a.b).c = a.(b.c)$
6. Ley de involución: $\sim\sim a = a$
7. Leyes de De Morgan: Permiten transformar sumas en productos y productos en sumas.
 - a. La negación de la suma de dos variables puede expresarse como el producto de las variables negadas
$$\overline{a + b + c + d + \dots} = \bar{a} . \bar{b} . \bar{c} . \bar{d} . \dots$$
 - b. La negación del producto de dos variables puede expresarse como la suma de las variables negadas.
$$\overline{a . b . c . d . \dots} = \bar{a} + \bar{b} + \bar{c} + \bar{d} + \dots$$

Circuitos lógicos combinacionales

Un circuito combinacional permite que en las salidas se obtengan valores binarios, el cual depende únicamente de los valores de entradas.

Un circuito se representa con un diagrama en bloque donde se especifican las n entradas y las m salidas. Las entradas representan los requerimientos de un problema y cada una puede asumir un valor verdadero si el requerimiento se cumple o falso en caso contrario.

Circuito generador de paridad

Permite la detección de un error en la transmisión de información entre un emisor y un receptor. Ambos se asocian a un generador de paridad que controla la cantidad total de 1 en la transmisión. Si el circuito es de paridad par contabilizará que la cantidad de 1 sea par e impar en el caso del circuito generador de paridad impar.

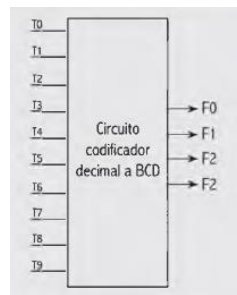
Circuito comparador de magnitudes

Permite comparar dos números binarios de n bits para saber si estos son iguales, o si uno es mayor o menor que otro. El circuito tendrá n entradas que representan los bits a comparar y tres funciones de salida que serán X si a es menor que b, Y si a es igual a b y Z si a es mayor que b.

Tabla 6-2. Tabla de verdad de un circuito comparador de magnitud				
a	b	Z	X	Y
		$a > b$	$a < b$	$a = b$
0	0	0	0	0
0	1	0	1	1
1	0	1	0	1
1	1	0	0	0

Circuito codificador

Permite generar códigos a partir de señales binarias como entrada transformándolas a su equivalente en decimal.

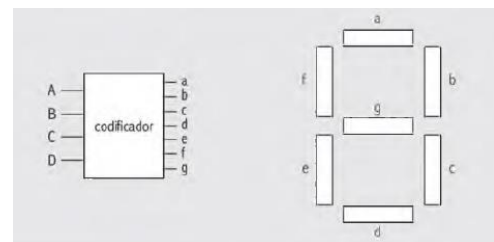


Circuito decodificador de Código

Al circuito ingresan combinaciones binarias pertenecientes a alguna convención de representación y son convertidas a señales binarias para representar en algún dispositivo de salida de información.

- Decodificador BCD a display de 7 segmentos: Para cada segmento un 1 activa un led y un 0 lo vuelve inactivo.

Tabla 6-5. Tabla de verdad											
A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	1	1	1	1	1	1	0	
0	0	0	1	0	1	1	0	0	0	0	
0	0	1	0	1	1	0	1	1	0	1	
0	0	1	1	1	1	1	1	0	0	1	
0	1	0	0	0	1	1	0	0	1	1	
0	1	0	1	1	0	1	1	0	1	1	
0	1	1	0	1	0	1	1	1	1	1	
0	1	1	1	1	1	1	0	0	0	0	
1	0	0	0	1	1	1	1	1	1	1	
1	0	0	1	1	1	1	0	0	1	1	
1	0	1	0	X	X	X	X	X	X	X	
1	0	1	1	X	X	X	X	X	X	X	
1	1	0	0	X	X	X	X	X	X	X	
1	1	0	1	X	X	X	X	X	X	X	
1	1	1	0	X	X	X	X	X	X	X	
1	1	1	1	X	X	X	X	X	X	X	



Para completar el circuito decodificador a 7 segmentos es necesario hallar las funciones mínimas de a,b,c....g y confeccionar el diagrama de lógica que incluye a todas las funciones.

Circuito multiplexor

Circuito cuya función digital es seleccionar entre 1 de las 2^n entradas, una única salida. La línea de entrada se elige a partir de los valores que pueden tomar las n líneas de selección.

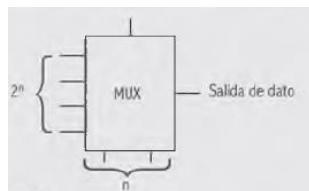


Tabla 6-9. Tabla de verdad

a	b	F
0	0	$D0$
0	1	$D1$
1	0	$D2$
1	1	$D3$

Circuito demultiplexor

Es un circuito digital que conecta la entrada con una de las 2^n salidas. Las n entradas de selección determinan cual es la salida encargada de transmitir el dato.

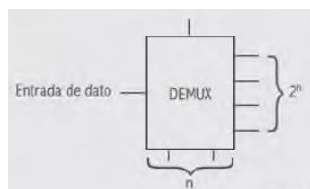


Tabla 6-8. Tabla de verdad

a	b	$F0$	$F1$	$F2$	$F3$
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

Circuitos semisumadores

Circuito cuya función es sumar 2 bits sin tener en cuenta el acarreo anterior. Tiene dos salidas, una que representa la suma y la otra que representa el acarreo.



Tabla 5-9. Tabla para la suma de 2 bits.

a_0	b_0	s_0	c_0
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

La función s responde a la compuerta XOR
La función c responde a la compuerta AND

Circuito sumador total

Tiene como finalidad sumar dos bits y el acarreo anterior, generando como salidas el resultado de la suma y el nuevo acarreo.

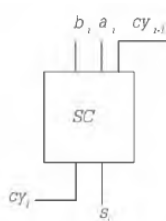


Tabla 5-10

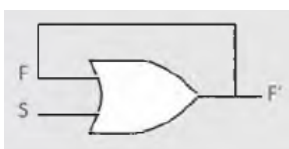
a_i	b_i	cy_{i-1}	s_i	cy_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Circuitos lógicos secuenciales

Los circuitos secuenciales producen salidas que dependen del valor de sus entradas y del estado que tenían en un instante anterior. En la representación de estos circuitos debe tenerse en cuenta que las funciones de salida realimentan las entradas.

Flip-flops (biestables)

Es un arreglo de compuertas capaz de almacenar un bit. Tiene dos salidas, una para el bit almacenado y otra que representa su complemento. Solo tiene dos estados de funcionamiento posibles, permaneciendo en cualquiera de ellos si los niveles lógicos de entrada no fuerzan su cambio.



Los flip-flops se interconectan para formar circuitos lógicos secuenciales para almacenar datos, temporización, conteo y secuenciamiento.

Para que un circuito sea un elemento de memoria, tiene que cumplir con dos condiciones:

1. Retener un bit, aún después de la eliminación de sus entradas.
2. Poder ser puesto a 0 o a 1

Flip-flop RS asincrónico

Tabla 6-12. Tabla de verdad

R	S	Qt	Qt+1
0	0	0	0
0	0	1	1
0	1	1	1
0	1	1	1
1	0	0	0
1	0	0	0
1	1	0	X
1	1	1	X

R es la entrada de reseteo o puesta a 0.

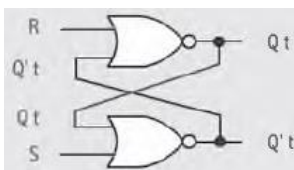
S es la entrada de seteo o puesta a 1.

Qt el almacenamiento del valor del flip-flop antes de activado.

Qt+1 es el almacenamiento del estado posterior.

Q't El complemento del valor del flip-flop antes de ser activado.

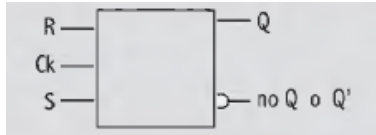
Q't+1 el complemento del almacenamiento del estado posterior a ser activado.



Las combinaciones con salidas Qt+1 en X son combinaciones prohibidas, ya que provocarían que ambas salidas sean iguales y no se cumpliría que $Q \neq \sim Q$

Flip-flop RS síncrono

La presencia de un pulso de reloj determina el instante a partir del cual las entradas modifican su estado.



R	S	Q(t+1)
0	0	Qt
0	1	Set
1	0	Reset
1	1	X

Flip-flop JK

El flip-flop JK conmutará cuando ambas entradas estén en 1 (es decir en la salida del instante posterior se almacenará el estado opuesto al instante inicial), estará en modo set cuando $J=1$ y $K=0$, en estado reset cuando $K=1$ y $J=0$ y se mantendrá en el mismo estado cuando $J = K = 0$.

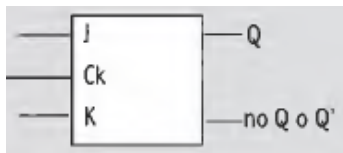


Tabla 6-13. Tabla de verdad de un biestable J-K sincrónico

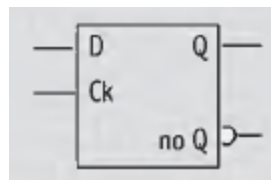
J	K	Qt	Qt+1
0	0	0	0
0	0	1	1
0	1	1	0
0	1	0	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Flip-flop D

Se construye a partir del JK unificando las entradas J y K negada de manera que cuando estas son distintas el circuito se comporta según la tabla de verdad. El valor que entra, es el valor que sale del circuito.

Tabla 6-16. Tabla de verdad

D	Qt	Qt+1
0	0	0
0	1	0
1	0	1
1	1	1

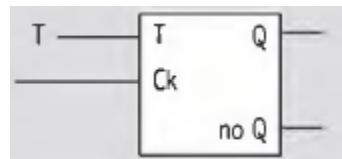


Flip-flop T

Es un derivado del flip-flop JK considerando que las dos entradas son iguales. Las salidas de este circuito pueden ser dos: O se mantiene el valor del estado anterior ($t=0$) o conmuta al estado contrario en el siguiente intervalo ($t=1$).

Tabla 6-14. Tabla de verdad

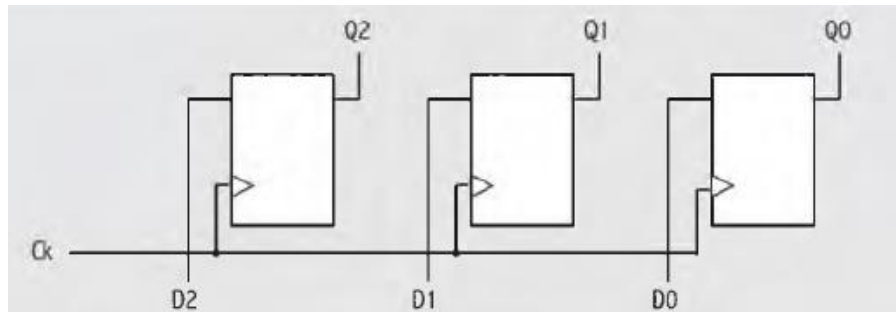
J	K	Qt	Qt+1
0	0	0	0
0	0	1	1
0	1	1	0
0	1	0	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0



Registros de desplazamiento

Es una función digital que suele utilizarse para retener información binaria temporalmente. Están formados por uno o más flip-flops.

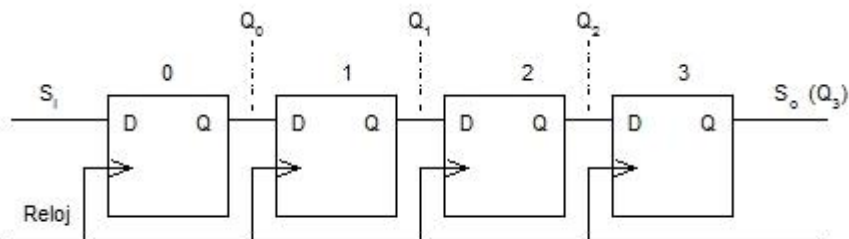
Paralelo-Paralelo



Su función es transferir la información que viene por las líneas de entrada en paralelo a salidas en paralelo.

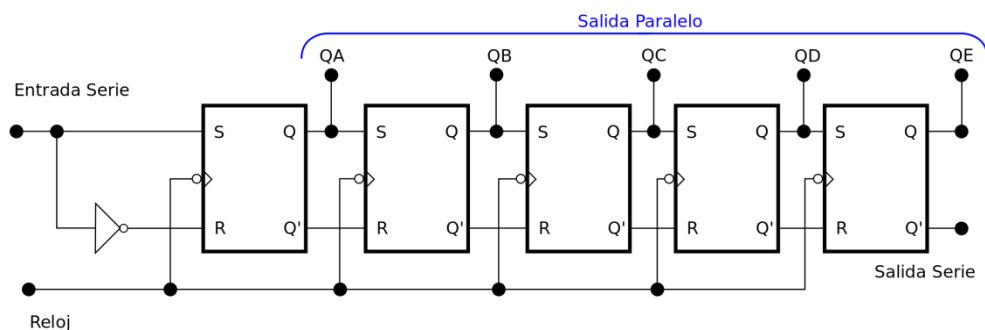
Serie-serie:

Con la señal de reloj el bit almacenado en el primer biestable se desplazará al siguiente y así uno tras otro hasta la salida en serie. Esto sucede así porque la salida Q está conectada a la S del siguiente biestable



Serie-paralelo

Se le añade una salida a cada una de las salidas Q del biestable: de esta manera se pueden obtener todos los datos a la vez.



Registros contadores

Tienen la característica de contar eventos en orden ascendente o descendente. Dependiendo de la cantidad de biestables que lo componen, un contador ascendente contara de 0 a 2^{n-1} eventos y un contador descendente contara de 2^{n-1} hasta 0. En ambos casos los contadores vuelven a su estado inicial una vez realizada la cantidad n de conteos.

Unidad III: Tecnología

Memorias del sistema

La memoria principal

La memoria almacena bits en celdas independientes. Cada celda se identifica y localiza mediante un número binario que se denomina dirección o posición de memoria. La información que almacena cada celda puede referirse a instrucciones o datos. En cada celda de memoria solo pueden leerse o escribirse 8 bits por vez, sin posibilidad de operar menor cantidad de bits, o un bit aislado. Si los datos o instrucciones ocupan más de un byte, se almacenan fragmentados en varios bytes que estarán contenidos en celdas consecutivas de memoria.

Clasificación de memorias

Las memorias se clasifican según ciertos parámetros:

- Modo de acceso a la unidad de información
- Las operaciones que aceptan por cada acceso
- La duración de la información en el soporte

Clasificación según el modo de acceso a la unidad de información

Memoria de acceso aleatorio

Cuando un componente de selección habilita una palabra e inhabilita las demás. El tiempo de acceso es independiente del lugar físico que ocupa la unidad de información en el soporte.

Memoria de acceso secuencial

Cuando para acceder a la unidad de información se establece una posición de referencia, a partir de la cual comienza un rastreo de la unidad de información que consiste en la lectura de todas las unidades que la precedan, hasta lograr la búsqueda. El tiempo de acceso depende de la distancia entre la posición inicial y la unidad de información.

Memoria de acceso asociativo

es cuando la búsqueda de la unidad de información implica la comparación de un grupo de bits de la unidad de información con el contenido de una posición de memoria.

Clasificación según las operaciones que se aceptan por cada acceso:

Memoria de lectura/escritura:

Admite ambas operaciones.

Memoria de solo lectura:

Admite solo esta operación.

Clasificación según la duración de la información:

Memorias volátiles:

Pierden su información con el corte de suministro de la corriente eléctrica

Memorias no volátiles, permanentes:

No pierden información con el corte de suministro eléctrico.

Memorias RAM:

Es sinónimo de memoria de lectura y escritura y de memoria volátil (pierden la información almacenada cuando se corta el suministro de energía eléctrica). Las memorias actuales son del tipo DRAM. La palabra dinámica identifica que en cada celda de memoria se almacena un byte, que necesitan ser periódicamente recargados eléctricamente.

Memoria ROM:

Memoria de solo lectura. Son memorias de acceso aleatorio, cuya escritura demanda muchísimo más tiempo que su lectura. Son no volátiles, almacenan información en forma permanente sin necesidad de energía eléctrica para guardar los datos, pero sí para leerlos. Contienen el programa para el arranque inicial de las computadoras y los programas que sirven para tratamiento de entrada y salida de datos.

Memoria principal es RAM + ROM.

La característica en común entre ambas es que se puede acceder a su contenido de manera aleatoria. El tiempo de acceso a los datos es menor en la RAM. El contenido de cada celda de las RAM puede ser leído o escrito cuantas veces se quiera, mientras que en las ROM se accede al contenido de una posición de memoria solo para leerla.

¿Qué significa que el acceso a la memoria principal es al azar (Random)?

Implica que cualquier posición puede encontrarse sin tener que recorrer las $n-1$ posiciones anteriores. El tiempo de acceso es el mismo para cualquier dirección sin importar su número.

Proceso de acceso aleatorio a una celda de memoria:

Los n bits de la dirección formada en el registro de direcciones viajan por las líneas del bus de dirección hasta las entradas del circuito decodificador unido a la memoria. A cada celda de memoria llega una salida del decodificador, solo una de ellas estará habilitada y será a la que se accederá. Cualquiera de las 2^n salidas del decodificador tarda el mismo tiempo en ser habilitada (acceso random).

¿Qué es capacidad de memoria y que son las unidades KB, MB y GB?

Capacidad de memoria

Es la cantidad total de bytes que puede guardar. En el presente, lo común es que en cada celda se guarde un byte de información. Entonces una memoria con n celdas tendrá capacidad de n bytes. El número de celdas puede expresarse en kilobytes, megabytes, gigabytes, terabytes, etc.

Bit

La capacidad medida en bits se utiliza para los registros, que son los soportes de información de menor capacidad.

Byte

Es aplicable a los registros. Las capacidades de memorias mayores se expresan en unidades que agrupan bytes

El número de celdas es siempre una potencia de dos y se redondea hacia abajo como convención.
La unidad más pequeña de almacenamiento de información es un byte, luego KB, MB, GB, TB.

Memorias RAM estáticas y dinámicas

Memorias SRAM

Son memorias estáticas y volátiles. Cada celda puede modificarse mediante el proceso de escritura, sino permanece inalterable siempre que esté conectada al suministro de energía.

Memorias RAM dinámicas

Son de lectura/escritura de bajo costo y gran capacidad de almacenamiento. Son más lentas que las de tecnología estática.

La decisión de incluir memorias dinámicas en el sistema, se debe a que utilizan menos componentes posibilitando el almacenamiento de mayor cantidad de bits en un mismo espacio y permitiendo aumentar la capacidad sin incrementar su costo global.

Las memorias dinámicas pueden considerarse menos apropiadas para responder con eficiencia a las exigencias de un procesador. La mayoría de los sistemas incluye uno o varios niveles de memoria caché ubicados entre la CPU y la DRAM.

Este tipo de memorias deben ser refrescadas periódicamente para evitar la pérdida de información.

Este procedimiento de "llenado", es físico y se denomina refresco (refresh). Consiste en recargar los condensadores que tienen almacenado un 1 para evitar que la información se pierda a causa de las fugas en los condensadores.

Memoria DRAM

El término dinámica hace referencia a que estas memorias degradan su información con el transcurso del tiempo, aun cuando están conectadas a la fuente de suministro. Por lo tanto, antes de que la información se pierda hay que restablecer la carga. Este proceso de regeneración se denomina ciclo de refresco (refresh cycle) y es un procedimiento físico que se produce a intervalos regulares.

Son memorias más lentas que las SRAM, pero tienen mayor capacidad.

Memoria RAM con acceso directo

Para permitir el acceso a la información de manera random, una memoria se organiza de manera matricial en filas y columnas. Cada fila o palabra de memoria o posición tiene igual cantidad de bits (q) para toda la matriz. Las columnas están formadas por los bits de igual peso de todas las filas.

Para acceder a cada palabra de memoria, cada una de ellas se identifica con su número de fila dentro de la estructura matricial. El número que identifica la palabra en un acceso random o al azar se denomina dirección física.

RAM con acceso asociativo

Las memorias asociativas son accesibles por contenido. El contenido buscado se denomina rótulo, descriptor o argumento. Requiere que todas las celdas de almacenamiento se relacionen con circuitos que permitan la comparación, razón por la que se vuelven más caras y su uso se justifica en aplicaciones en las que sea imprescindible la búsqueda por contenido.

Controlador de memoria dinámica

En este tipo de memorias se multiplexan las direcciones, de manera que para acceder a una celda se considera una parte de la dirección física para acceder a la fila y otra parte para acceder a la columna en la matriz. Estas señales se denominan RAS y CAS.

El acceso al chip de las memorias dinámicas, se administra por un dispositivo hardware llamado "controlador de memoria dinámica".

¿Qué es el controlador de memoria?

Es un chip programable que está en la motherboard que cumple las siguientes funciones:

1- Divide la dirección emitida por la UCP en dos mitades (para direccionar una fila y una columna de la DRAM) enviándolas en forma sucesiva y en su correcta temporización a la Dram y avisa cuando la información esta direccionada en el bus. El controlador debe activar primero la señal RAS, y luego activará la señal CAS. El tiempo que permanezca activa la señal CAS será el utilizado como parámetro para indicar el tiempo de acceso. (Latencia CAS)

Latencia CAS: Numero de ciclos de reloj que transcurren desde que se realiza la demanda de datos hasta que estos se ponen a disposición del bus de datos.

2- Realiza el manejo de los bancos de memoria a fin de minimizar el tiempo de acceso cuando se direccionan posiciones sucesivas de la DRAM

3- "refresca" periódicamente las celdas de memoria a fin de reponerles su carga eléctrica.

Memorias ROM

Chips ROM

Son fabricados a medida, indicándole al fabricante el contenido que debe tener en cada celda. Ese contenido es el que tendrá para siempre.

Chips PROM

Se fabrican en serie, pero el interior de cada chip está preparado para que quien los utilice pueda escribir una sola vez los contenidos que tendrán sus celdas mediante un dispositivo electrónico. Una vez grabadas, no pueden ser re-escritas.

EPROM

Presentan una ventanilla transparente en su cara superior que permite cambiar el contenido de todas las celdas reescribiéndolas. Esto se hace borrando el contenido de todas las celdas haciendo pasar luz UV por la ventanilla durante unos 15 minutos.

EEPROM

Permite el borrado de celdas para reescribirlas sin sacar la pastilla del circuito. Esto puede hacerse unas 10.000 veces.

Memorias caché

Son memorias de tecnología de semiconductores de tipo SRAM, cuya velocidad de respuesta se ajusta de manera muy favorable a los tiempos del procesador. Su velocidad es compatible con las necesidades de obtención de información por parte del procesador.

NECESIDAD DE LA CACHÉ

Un procesador ejecuta cada instrucción en 5 etapas, de las cuales 3 consisten en un acceso a la memoria principal ocupando un tiempo equivalente al de acceso a la memoria en cada etapa. Este hecho es la causa de un mal rendimiento.

Las cachés se necesitan para aumentar la velocidad de la memoria.

Las cachés son ultrarrápidas pero de poca capacidad y muy caras, por lo que no se pueden sustituir las DRAMs por cachés. Debemos emplear la jerarquía de memoria, la cual consiste en interponer entre la CPU y la memoria DRAM una memoria ultrarrápida (caché).

La caché se utiliza como memoria intermedia entre el procesador y la DRAM y almacena en forma temporal la información a la que se accede con mayor frecuencia en esta última.

PRINCIPIO DE FUNCIONAMIENTO DE LA CACHE

Una caché está estructurada en tres bloques importantes:

1. BLOQUE DE ETIQUETAS: Es una memoria de acceso por contenido. No se accede por dirección de memoria, sino que, se compara el valor o dato con los que hay dentro de la memoria y así sabemos si se encuentra o no contenido en ella.

2. BLOQUE DE DATOS ASOCIADOS: Es un conjunto de datos de forma que a cada dato le corresponde una etiqueta.

3. LÓGICA DE CONTROL: Comparadores de "n" bits, tantos como tenga la etiqueta

La caché es un subsistema construido por una memoria de etiquetas, una memoria de datos y un controlador.

El controlador

Gestiona su actividad. Es el cerebro de la caché. Cuando éste trae una instrucción de la memoria principal, también trae las próximas instrucciones y las almacena en caché. Esto es así porque hay una alta probabilidad de que la próxima instrucción también se necesite. Esto incrementa la posibilidad de que la CPU encuentre la instrucción que precise en la memoria más rápida y, por lo tanto, pueda optimizar el tiempo

Caching: Procedimiento que anticipa las necesidades de posiciones de la memoria principal calculando la probabilidad de uso y criterios de vecindad espacial y temporal. (Cuanto mas vieja sea una referencia a dato, menor será la posibilidad de que se acceda a ella en lo inmediato).

Memoria de etiquetas

Es una matriz asociativa de $m \times n$, donde cada fila está asociada a una línea de **la memoria de datos** y se denomina etiqueta. Hay tantas etiquetas como líneas. El contenido de la etiqueta referencia el número de bloque de la memoria principal y que se almacenó en caché. Cada etiqueta hace referencia a una línea.

El bus de direcciones va a la caché, que tiene dos partes: la de las etiquetas y la de datos. Se comparan las n etiquetas con los bits de mayor peso de la dirección. Si alguna coincide, el comparador devuelve un 1, lo que implica presencia. En cambio, si no coincide ninguna, el comparador devuelve un 0 lo que implica ausencia, significa que esa dirección no está contenida en la caché. Hay un dato por etiqueta. Si el comparador devuelve un uno, el dato correspondiente a la etiqueta es transferido a la CPU reflejando los bits restantes, la posición en la que se encuentra el dato. Si el comparador devuelve un cero (ausencia) se accede a la memoria principal

Conexión en serie:

Es la más utilizada en los procesadores actuales, Debido a que el acceso a caché se realiza por un bus separado. La CPU sólo se conecta con la caché por lo que todas las peticiones que hace la CPU al bus del sistema son a través de la memoria caché. Todo lo que necesita la CPU del sistema se lo proporciona la memoria caché, cada vez que el dato está almacenado en la caché, el tiempo de acceso es muy reducido y evita manejar el bus del sistema.

Si la memoria caché contiene el dato solicitado por la CPU, en pocos nanosegundos proporcionará la información. Si el bus del sistema está desocupado, los módulos de entrada/salida pueden estar trabajando con la memoria principal. Se permite el paralelismo.

El inconveniente de la conexión en serie es la penalización de tiempo ya que cuando la información que necesita la CPU no está en la caché tiene que trasladar la petición al bus del sistema para poder acceder a la memoria principal.

Otro inconveniente es que la caché es de uso obligatorio; no se puede desconectar la caché y conectar la CPU al sistema.

Conexión en paralelo

Todo depende del bus de sistema. Cada vez que la CPU realiza una petición, la envía simultáneamente a la caché y a la Memoria Principal. Si se encuentra la información contenida en la caché es entregada al bus en pocos ns avisando a la memoria principal para que no continúe con la búsqueda de la información, parando el ciclo. Si la memoria caché no tiene el dato pedido, la memoria principal sigue trabajando, con lo cual no hay penalización de tiempo, lo que es una gran ventaja.

Se utiliza mucho porque cuenta con otra ventaja, la caché es opcional, se puede añadir o eliminar sin alterar el funcionamiento del sistema.

El inconveniente es que la CPU realiza todas las peticiones por el bus del sistema, quedando sobrecargada por el continuo flujo de información, lo cual deja muy poco espacio libre para E/S, ya que el bus del sistema se libera muy pocas veces

Traducción de la dirección fija

Como el tamaño de la memoria DRAM no coincide con la caché, sus respectivos espacios de direccionamiento son distintos, por lo tanto una dirección física deberá ser traducida o “mapeada” por el controlador que adapta la dirección y comprueba si existe tal referencia en caché. La traducción puede organizarse según tres formas de organización:

- Totalmente asociativa
- Asociativa de 1 vía de correspondencia directa

- Asociativa de conjunto o de n vías

Mapeo totalmente asociativo

A cada línea le corresponde una etiqueta. Se considera la mejor organización, dado que no existe una relación entre el identificador del bloque y su posición dentro de la caché.

Se puede organizar como un anillo, cada vez que se agrega un bloque, tanto las etiquetas como las líneas que ya estaban se desplazan un lugar. Si la caché ya estaba completa la primera de la cola se pierde (La más antigua).

Una desventaja es que su lógica de comparación es compleja y cara.

Mapeo asociativo de una vía o de correspondencia directa

La memoria RAM se divide en grupos del mismo tamaño e igual estructura que la línea de memoria de datos en caché. Hay una etiqueta por número sector RAM o línea caché.

Mapeo asociativo de n vías o de n conjuntos

Es similar al mapeo directo, pero cada línea admite n etiquetas y n matrices de datos, esto implica una caché n veces más grande y menor posibilidad de fracaso

Actualización de cache

El momento de actualizar una caché, es cuando se detecta una falla o ausencia de la palabra buscada, o sea, la palabra faltante está en la memoria principal. Si aún no se completó la caché, la escritura se realiza sin ningún tipo de sustitución; no obstante, por lo general ya se encuentran almacenados datos válidos con la consecuente necesidad de que el controlador asuma un criterio coherente para efectivizar el reemplazo.

En el caso de las organizaciones sectorizadas no hay otro criterio que el de reemplazar la única posición posible dentro de la caché Sin embargo, en las cachés con más de un nivel de asociatividad se utiliza una política de reemplazo o política de sustitución.

La asignación de una política de reemplazo afecta el rendimiento del subsistema de la misma forma que lo afecta su organización. El controlador ejecuta un algoritmo fijo y predeterminado, que se puede clasificar en por lo menos tres categorías:

1. LRU (Last Recently Used) La de uso menos reciente
2. FIFO (First In First Out) Primera en entrar, primera en salir
3. RND (Random) Aleatorio

LRU

Sustituye la información que hace más tiempo que fue referenciada Entonces, se debe considerar que parte de la información almacenada asociada a cada línea se utiliza para mantener atributos;

uno de esos atributos es un grupo de bits que representa esta característica y que el controlador utiliza para identificar la que lleve más tiempo sin que se haya accedido a ella, que no es necesariamente la más antigua.

FIFO

En este caso, las posiciones pueden desplazarse en el sentido de una cola, de modo que la primera que ingresó es la que primero sale, lo que es fácil de aplicar en una caché de tipo asociativo.

RND

En este caso el reemplazo se realiza al azar sobre cualquier línea. Como ventaja podemos interpretar que la aplicación del algoritmo es sencilla y, por lo tanto, rápida, pero su gestión puede caer en la contradicción de sustituir una, a la que se accedió en forma reciente.

Actualización de la memoria principal

Una modificación en cache implica la actualización de memoria. Las modificaciones a la información que gestiona el procesador se realizan en primer lugar sobre la caché. Esto es así porque se supone que es muy probable que se vuelva a acceder a aquello a lo que se accedió recientemente, según el criterio de vecindad temporal.

Debemos pensar que dos copias de la misma información en soportes distintos deben tener el mismo contenido, o sea, por cada actualización de la caché se actualiza la memoria principal. Sin embargo, en cierto sentido este procedimiento no es muy eficiente; la escritura en caché es rápida pero la que se realiza en la memoria es lenta. Por esta razón, también existen políticas de actualización de la memoria principal que se pueden clasificar en:

- Escritura inmediata
- Escritura obligada

Escritura inmediata

Este método es simple y consiste en actualizar de manera simultánea ambas memorias, de modo tal que no se genere incongruencia entre la información almacenada en ambos niveles. Esto reduce el rendimiento debido al tiempo empleado en la escritura sobre la memoria principal, con la consecuencia adicional de mantener el bus de dato ocupado por más tiempo. Sin embargo, son los subsistemas más económicos.

Escritura obligada

Este método admite sólo las actualizaciones estrictamente necesarias en memoria principal, o sea que cierta información alojada en caché puede actualizarse varias veces antes de ser efectivamente actualizada en memoria principal. Esto es razonable de acuerdo con el criterio de vecindad temporal. Sin embargo, el método genera incongruencia entre la información almacenada en ambos soportes. Esto puede ser grave si no se actualiza la memoria principal cuando se permite que cualquier otro dispositivo tenga acceso a ella. Para evitar un acceso a información obsoleta, antes de que el microprocesador conceda el acceso a otro dispositivo, se

habilita al controlador de caché para que copie las líneas marcadas como modificadas en la memoria principal.

Hay que actualizar la memoria obligatoriamente cuando:

1. Se accede a una posición de la memoria principal modificada en la caché por la CPU. Antes de dar paso a esa lectura, hay que escribir el dato modificado.
2. Cuando hay que eliminar una línea en la caché porque está llena, en la cual hay un dato modificado. Antes de borrarlo hay que enviar dicho dato a la memoria principal

Niveles de caché

En las computadoras actuales es común la utilización de varios niveles de caché:

- Caché de nivel 1: Es de tamaño reducido y se ubica funcionalmente, interceptando los datos que entran en el microprocesador desde la memoria o los que salen ya procesados hacia la memoria.
- Caché de nivel 2: Es de tamaño mayor y se ubica entre la caché de primer nivel y la memoria principal. Atiende pedidos menos frecuentes que la primaria, se utiliza como ampliación de la anterior. No solo contendrá la misma información que la almacenada en la primaria, sino unos Kbytes más.

Las ventajas de contar con varios niveles de caché, se notan cuando las aplicaciones son muy grandes o requieren acceso frecuente a los datos en memoria. Esto produce un alto tráfico en el bus, que se beneficia con la implementación de estos niveles de memoria.

Memorias auxiliares

Toda memoria alojada en una unidad de E/S se denomina memoria externa o secundaria. Son consideradas muy lentas con respecto a las memorias de semiconductores.

Las memorias externas se incluyen en toda computadora debido a su carácter no volátil, o sea que son imprescindibles para retener información cuando el sistema está desconectado de la fuente de alimentación.

Los dispositivos de E/S más utilizados para “soportar” estas memorias son el disco rígido y los dispositivos de tecnología óptica, como la unidad de DVD.

Los parámetros para tener en cuenta a la hora de la elección del soporte son:

- Costo por bit almacenado
- Tiempo de acceso
- Modo de acceso
- Capacidad de almacenamiento

- Tipo de tarea que utiliza la información.

Las memorias externas actúan de depósito y toda información que no se necesite para el proceso actual se almacena en ellas hasta que sea demandada.

Se denomina soporte al elemento físico que retiene los bits de información. Los soportes utilizados con mayor frecuencia son los medios magnéticos.

El tiempo de acceso a la información se mide en microsegundos y depende del tipo de unidad accedida. Dos parámetros que influyen en el tiempo de acceso son el tiempo de búsqueda en el soporte y el tiempo de transferencia entre el par memoria principal-memoria secundaria.

Archivos

Todo tipo de información se aloja en las memorias externas en una estructura denominada archivo. Un archivo se diferencia de los demás por el nombre simbólico que se le asigna en el momento de su creación. La forma de nombrar archivos la determina el sistema de archivos del sistema operativo de la computadora. Estas pautas están documentadas y se deben respetar, para que el archivo pueda ser reconocido por el sistema

Acceso a archivos

- Acceso secuencial: Un proceso que puede leer todos los bytes o registros del archivo en orden, comenzando desde el principio, sin saltar los registros fuera de ese orden.
- Acceso directo: cuando un proceso puede leer bytes o registros de un archivo fuera de orden o puede acceder a los registros considerando uno de sus campos como una clave. Esta técnica involucra un acceso al azar.

Atributos de un archivo

El sistema de archivos asocia a cada uno cierta información adicional, por ejemplo, el día y la hora en que se creó y su tamaño. Esta información extra se conoce con el nombre de atributos del archivo.

Las siguientes son algunas de las posibilidades

- Protección: determina quién y de qué manera se puede acceder al archivo
- Sólo lectura: el archivo puede leerse, pero no modificarse.
- Sistema: es un archivo de uso exclusivo del sistema operativo.

Operaciones sobre archivos

Los diferentes sistemas operativos permiten realizar operaciones para almacenar y recuperar la información en los archivos. Estas operaciones se ejecutan mediante llamadas al sistema.

- Create (crear): permite crear el archivo sin datos para asignarle alguno de sus atributos
- Delete (borrar): permite borrar el archivo y liberar el espacio ocupado en el soporte.
- Open (abrir): todo archivo debe ser abierto por el proceso antes de ser leído o escrito.

- Close (cerrar): todo archivo que deja de utilizarse debe cerrarse para liberar su tabla de acceso en memoria principal.
- Read (leer): permite leer el archivo
- Write (escribir): permite escribir el archivo
- Append: permite escribir el archivo sólo a partir del indicador de final de archivo, esto es, a partir del último byte grabado.
- Rename (renombrar): se utiliza para cambiar el nombre actual del archivo existente

Sistema de archivos en discos de tecnología magnética

La necesidad de almacenar información permanente en los soportes se debe a que:

- Todos los procesos en una computadora necesitan almacenar y recuperar información y es común que ella deba sobrevivir a la terminación del proceso
- Es necesario almacenar grandes volúmenes de información, por lo tanto, se justifica la necesidad del uso de memorias externas.
- Muchos procesos pueden necesitar el acceso a la misma información en forma simultánea, por lo tanto, cuando un proceso modifica cierta información, ésta debe estar protegida del acceso de otro.

Disco magnético desde el punto de vista lógico

En la actualidad los discos magnéticos están constituidos por platos rígidos, de allí su denominación disco rígido, para diferenciarlos de los ya extinguidos discos flexibles. Están dispuestos en unidades de cabezas fijas o móviles, en grupos de uno o más platos.

Un soporte en un disco está constituido por un plato de aluminio, recubierto de material magnetizable con forma de disco, que gira alrededor de un eje dispuesto en la unidad, al que se accede por medio de una o más cabezas lectograbadoras.

La forma en que las cabezas recorren la superficie del plato, describen círculos concéntricos denominados pistas.

Una pista es una división lógica y no física de la superficie del soporte, que se produce por efecto de la acción de rotación del soporte y de la posición fija de la cabeza al momento de grabación o lectura.

Si la unidad graba las dos caras de un plato o cuenta con más de uno, habrá muchas pistas de igual número (a la misma distancia respecto del eje) en las distintas caras. Por cada cara grabable habrá una cabeza lectograbadoras. Todas ellas acceden de manera simultánea a un cilindro en particular, cuya identificación está dada por un número relativo que tiene que ver con la distancia del radio entre las cabezas y el eje.

El concepto de cilindro, es importante, dado que la información se graba en sentido vertical, o sea que se completa un cilindro y luego se posiciona el brazo en el siguiente. Cada movimiento de

cabeza es mecánico, por lo tanto resulta óptimo grabar todas las pistas de igual número sin modificar la posición del brazo que sostiene las cabezas.

Cada pista está dividida en sectores que se pueden identificar con un número.

El número de sectores por pista es fijo y, debido a que las pistas externas tienen una circunferencia mayor que las internas, estas últimas son grabadas a mayor velocidad.

El número de sectores que se pueden definir en la pista más interna limita el número de sectores por pista para todo el plato.

Si bien en la mayoría de los casos la unidad de referencia que hace el proceso para acceder a un archivo es el registro lógico, la unidad de acceso al soporte es una agrupación de ellos, denominada bloque.

En un disco por cada bloque se graban uno o varios sectores; cada sector es una porción de pista y, en términos del sistema operativo Windows, un bloque se denomina cluster.

Todos los sectores de una pista almacenan la misma cantidad de bits

A su vez los sectores de pistas distintas observan el mismo criterio, se deduce que la densidad de grabación aumenta en forma sucesiva respecto de la distancia del eje central.

La cantidad de bits por pulgada depende de las cabezas lectograbadoras, la calidad del medio de grabación y las revoluciones por minuto a las que gira el disco.

La cantidad de pistas por pulgada depende de las cabezas lectograbadoras, la precisión mecánica con la que se pueden posicionar las cabezas y la aptitud del disco de girar formando un círculo muy definido.

La determinación de la cantidad de información por sector o por bloque tiene relación con la capacidad total del volumen.

El sistema de archivos tiene la responsabilidad de aportar una estructura lógica, que permita un reconocimiento ágil de la ubicación de la información por acceder. Estos programas se dedican a organizar y administrar la gestión de almacenamiento, permitiendo la captura de bits desde el disco. Se dice que un programa que registra en el soporte una estructura productiva realiza un proceso de "dar forma" y es conocido como formateo o inicialización del soporte.

Los parámetros que determinan la ubicación de una entidad para su acceso son el número de pista o cilindro, el número de cara y el número de sector. El formateo es el nombre con que se denomina al comando que ejecuta el programa de inicialización de un disco y que genera la grabación de marcas, que identifican estos parámetros. De este modo, se puede localizar el lugar donde se graban los archivos. El proceso de formateo es, entonces, imprescindible para trabajar con el soporte.

Cuando se ejecuta el comando que implica abrir un archivo existente, se determina dónde se encuentra éste en el disco. Para esto es necesaria la identificación de cilindro, cabeza y sector. Luego la controladora del disco lleva al brazo actuador, a posicionar las cabezas sobre la pista adecuada. A medida que el disco rota, la cabeza apropiada lee la dirección de cada sector en la pista. Cuando aparece el sector deseado bajo la cabeza lectograbadora, se lee todo el contenido del sector en forma secuencial. Esta lectura vuelca los datos necesarios en una memoria ultrarrápida especial, denominada caché de disco. Entonces, el controlador que actúa de interfaz de la unidad de disco envía la información necesaria a la memoria principal de la computadora

Particiones

Un disco rígido es una unidad física, sin embargo, los sistemas operativos las consideran unidades lógicas. En un mismo disco físico se pueden crear dos o más unidades lógicas, a las que se denomina partición. Desde el punto de vista del software, cada partición es un disco lógico o unidad lógica diferente. La partición se crea cuando se da formato al disco, su capacidad es fija y está constituida por un grupo de cilindros contiguos.

Una de las razones por las cuales se crea más de una partición, es para que en cada una se instale un sistema operativo distinto; de esta manera, se crean dos entornos de trabajo diferentes para una misma computadora.

En un mismo disco puede haber particiones primarias y secundarias. En una partición primaria se instala un sistema operativo, sólo una de ellas está activa por vez. Una partición secundaria se utiliza para almacenar archivos que, en algunos casos, pueden ser accedidos por distintos sistemas operativos, pero no contienen los programas de instalación.

File allocation system (FAT)

Es una tabla de asignación de espacio para los archivos del área de datos. En cada entrada de la tabla se representa un cluster del área de datos y esta entrada a su vez contiene un número que actúa de puntero a otro cluster. La función de la FAT es mostrar la cadena de clusters en la que se aloja el archivo. Inicialmente los archivos se almacenan en clusters sucesivos, pero cuando el usuario elimina ciertos archivos quedan huecos que podrán ser utilizados por otro nuevo. Es común que el hueco no tenga el tamaño suficiente como para alojar el archivo nuevo y, por lo tanto, se produzca un salto al próximo cluster vacío, de manera que permita alojar el resto de los datos que se desea almacenar. Este procedimiento se repite una y otra vez y los archivos quedan fraccionados en el soporte.

Si bien es lógico y deseable que los archivos aprovechen los huecos vacíos del soporte, recuperar un archivo fraccionado en clusters ubicados en cilindros diferentes insume más tiempo, porque provoca el movimiento mecánico del brazo, por lo que es mejor que un archivo se aloje en clusters sucesivos. En este tipo de organizaciones se produce una reestructuración de la información que se conoce con el nombre de desfragmentación.

El directorio raíz y los subdirectorios

El directorio es un archivo que se actualiza en relación con las operaciones que se realizan con los archivos. Es una tabla con información propia de los archivos y a cada uno se asigna una entrada que básicamente indica su nombre y atributos.

Para acceder a un archivo guardado en un disco, se accede al directorio y se lo ubica por su nombre de archivo en el disco y un número del cluster de comienzo, que permita su ubicación en el soporte y el acceso a la primera entrada de la FAT, para conseguir su cadena de locaciones. Además de archivos, un directorio puede contener otros directorios, que dependen del primero. La información que se puede encontrar en una entrada se resume así:

- Nombre del archivo
- Byte de atributos
- Hora de última modificación
- Fecha de última modificación
- Número de cluster donde comienza el archivo
- Tamaño del archivo

Es importante detenerse en los atributos de archivo. Los archivos admiten diferentes operaciones, las cuales pueden ser reguladas por los atributos asociados al archivo. El estado de un atributo puede representarse con un bit.

El procedimiento de acceso requiere ciertas operaciones

Creación

Se accede al directorio comparando el nombre del archivo con cada una de sus entradas. Si el archivo existe, no podrá crearse. Como consecuencia, deberá existir la opción de reemplazo de no por el otro, previa consulta al usuario. Si el archivo no existe, se le asigna una entrada al directorio se busca en la FAT. La primer entrada con contenido 0 se asigna como primer cluster, se graba y se busca otra entrada vacía; así, se arma una cadena que puede estar físicamente discontinua hasta la finalización del archivo.

Lectura/grabación

Se accede al directorio comparando el nombre del archivo por leer o grabar. Si se quiere leer y no existe, provoca un error archivo no encontrado [file not found]; si existe, se accede a los clusters de datos, consultando alternativamente la FAT.

Eliminación de un archivo

Se accede al directorio comparando el nombre del archivo con cada una de sus entradas. Si no se produce equiparamiento, se genera un mensaje del error; en caso contrario, se marca la entrada del directorio indicando que ésta queda libre; a su vez, se liberan las entradas de la FAT que le estaban asignadas. El archivo no se borra físicamente del soporte, sino que los territorios

ocupados quedan liberados gracias a la actualización del directorio y las entradas de la FAT regrabadas con el número 0.

Lo más lógico en todos los soportes es que el área de sistema se aloje en el cilindro que permita el menor tiempo de acceso a las cabezas lectograbadoras, dado que éstas siempre son consultadas para acceder al soporte. Éste es el cilindro cuya distancia física de las cabezas es menor, mientras que los clusters de datos ocupan el resto de los cilindros.

Sistema de archivos N T F S

Es un sistema de organización de archivos que ordena la información almacenada en el soporte de manera distinta a los que utilizan FAT y es más eficiente para particiones de gran tamaño. Las características que los diferencian son:

- Aprovecha mejor el espacio en disco.
- Reduce la fragmentación de archivos típica del sistema FAT.
- Compatibilidad mejorada con los metadatos

Los metadatos son archivos creados en forma automática por el sistema cuando formatea un volumen de NTFS. Se localizan utilizando una tabla de archivo maestra (MFT). Esta tabla es el punto de arranque de cualquier unidad bajo NTFS y se la puede analogar a la FAT, pero es mucho más que una lista de clusters vacíos.

Por cada archivo o directorio que se crea en el disco, se genera un registro en la MFT. Por cada archivo o directorio que se crea en el disco, se genera un registro en la MFT, tan pequeño que los datos y los atributos de los datos entran en un registro, queda almacenado en la MFT y no requiere espacio adicional en el área de datos.

Sistemas de archivos en discos de tecnología óptica

El primer sistema de archivos creado para medios ópticos denominados CD-ROM, esto es, que no aceptaban reescritura. Se trataba del primero que soportaba archivos para diferentes sistemas operativos. En tecnología óptica ésta es la característica más buscada, puesto que, en general, se utiliza el medio no sólo en distintas computadoras con sistemas operativos diferentes sino también en múltiples dispositivos digitales. En la actualidad, el formato más utilizado es el UDF que mejora la anterior, ya que permite la grabación en medios ópticos. Sus siglas significan formato de discos universal, y permite el acceso a los soportes ópticos desde diferentes sistemas operativos. Es, pues, obligatorio para DVD y admitido por los CD.

Dispositivos de entrada y salida. Generalidades

La CPU intercambia instrucciones y datos con distintos periféricos, como la unidad de disco rígido, una tarjeta de memoria o un DVD. Estos periféricos se denominan dispositivos de entrada salida y están conectados a la CPU, en la mayoría de las computadoras, con un bus denominado bus de entrada/salida. Las computadoras permiten el acceso a muchos de estos dispositivos.

Discos rígidos

Son los dispositivos de E/S de mayor uso en nuestros días. Un disco está constituido por una base, recubierta de material magnetizable con forma de circunferencia que gira alrededor de un eje dispuesto en la unidad y que es accedido por una o más cabezas lectograbadoras. Una cabeza registra los bits en círculos concéntricos denominados pistas.

Una pista es una división lógica y no física de la superficie. Si la unidad graba las dos caras de un plato o cuenta con más de uno, o ambas; habrá muchas pistas de igual número en las distintas caras. Por cada cara grabable habrá una cabeza lectograbadora que accede en simultáneo a cada una de ellas. Esto es importante dado que la información se almacena de manera vertical. Esto resulta óptimo ya que se grabarán todas las pistas de igual número sin modificar su posición.

Controladora de disco

La electrónica del microprocesador y la memoria asociada de un disco es compleja y delicada, el conjunto de platos está incluido en una caja soldada de aluminio que provee un entorno libre de contaminación para la operación de las cabezas lectograbadoras.

La controladora del disco está constituida por:

- Controladora del motor del eje y brazo actuador, para asegurar que los brazos giran a la velocidad correcta y que el brazo posiciona las cabezas lectograbadoras en el lugar preciso sobre el plato.
- Controladora de interfaz, para comunicarse con la CPU del sistema de transferencia adecuado.
- Un microprocesador que ejecuta los comandos propios del disco, como estacionar cabezas o detener motor; y una memoria asociada para almacenar el bloque que se leyó o va a transferirse.

La cobertura magnética de una unidad de disco está compuesta por áreas dominios. Cada dominio es magnetizable. Los 0 y 1 que conforman la información en un disco se almacenan como patrones magnéticos en la cubierta magnética de éste. Para escribir información en un disco, una corriente eléctrica produce un patrón magnético en la superficie del dato a través de las cabezas. Para leer la información, las cabezas lectograbadoras convierten los patrones magnéticos almacenados en señales eléctricas que representan los datos almacenados.

Especificaciones técnicas de un disco

Hay especificaciones que permiten medir la funcionalidad mecánica de la unidad, otras, sus características electrónicas y otras sus requerimientos ambientales.

Especificaciones de partes mecánicas controladas electrónicamente:

- Tiempo de búsqueda
- Tiempo de cambio de cabezas (switch)

- Latencia rotacional
- Tiempo de acceso

Tiempos de acceso al disco

Tiempo de búsqueda

Es el tiempo que tarda el brazo en mover las cabezas lectograbadoras entre las pistas en milisegundos. Es una especificación valiosa y el tiempo que se toma como parámetro es el de búsqueda promedio, que es el tiempo que toma posicionar las cabezas lectograbadoras de la unidad del disco para una posición aleatoria.

Tiempo de cambio de cabezas o tiempo de switch

Mide el tiempo medio que le lleva a la unidad de disco cambiar entre dos de las cabezas cuando está leyendo o escribiendo datos.

Latencia rotacional

Tiempo que le lleva al disco girar el plato al sector correcto, una vez que la cabeza se posiciona sobre la pista adecuada. En el peor caso, la cabeza llega al cilindro justo después de que el sector deseado pasó por debajo de esta. En promedio, el disco necesita girar solo media vuelta hasta que el próximo sector para leer o escribir esté debajo de la cabeza.

Tiempo de acceso a los datos

Mide lo que se tarda en posicionar una cabeza sobre una pista particular y encontrar el o los sectores de interés dentro de esa pista para leer o escribir. Es una combinación del tiempo de búsqueda, de cambio de cabezas y la latencia rotacional. Se mide en milisegundos.

Una vez que la CPU genera un pedido de datos, la unidad de disco accede a los sectores apropiados, los lee y los almacena en la memoria caché. Una vez almacenados en la caché, pueden enviarse hacia la memoria interna.

Velocidades de transferencia

Tiempo que conlleva la transferencia de datos entre el disco y la memoria interna. Depende de dos medidas: La velocidad de transferencia del disco y la velocidad de transferencia del host. Se mide en megabits por segundo (Mb/S) o en gigabits por segundo (Gb/s)

Dispositivos de almacenamiento removibles:

Discos ópticos

Almacenan grandes cantidades de datos y se utilizan cabezales que emiten rayos láser para leer/escribir desde/en el medio. Si bien ofrecen gran capacidad de almacenamiento, no son tan veloces como el disco rígido.

Hay tres tecnologías de discos ópticos disponibles para almacenamiento. CD-ROM, DVD-ROM y DVD-RAM.

CD-ROM son la forma más popular de almacenamiento en disco óptico, sin embargo solo pueden leerse. En el presente (Del libro) casi todas las aplicaciones se distribuyen en CD-ROM debido a las posibilidades multimedia que brinda la capacidad de los CD-ROM.

El DVD ROM es un disco de video digital de solo lectura.

El DVD-RAM es un disco de video digital regrabable.

Discos magneto-ópticos

Estos sistemas combinan la tecnología de los medios magnéticos tradicionales, con la tecnología de los discos ópticos. Permiten empacar cientos de MB de datos en un disco que luce como un disquete.

Una unidad MO escribe en el disco usando una cabeza lectograbadora asistida por un láser. Debido a que un láser puede enfocarse en un campo más angosto que un campo magnético tradicional, los discos MO cuentan con una densidad de datos mayor que con tecnología de disco rígido.

Durante la operación de lectura, la unidad MO usa el láser para censar los datos almacenados en disco, recorriendo la superficie del disco. La unidad detecta una reflexión de luz por los bits de datos orientados en un sentido y no detecta reflexión por los datos orientados en sentido contrario. Así distingue entre 0 y 1 almacenados.

Tarjetas de memoria

Las tarjetas de memoria consumen poca energía y son compactas, confiables y livianas. Debido a su tamaño, facilitan el transporte de datos.

Tarjetas ROM y OTP

Estas tarjetas se comportan igual. La diferencia está en la manera en que se programa la información almacenada en la tarjeta.

Las ROM se fabrican y sus datos o programas van incorporados en sus chips. Las tarjetas OTP, se fabrican borradas, luego sus chips pueden escribirse con un dispositivo especial. Una vez escritos, la información almacenada no puede borrarse o alternarse.

Tarjetas SRAM

Tienen una ventaja respecto de las ROM y OTP: los usuarios pueden escribir, modificar o borrar la información almacenada en ellas. Sin embargo son volátiles, lo que obliga a que se las alimente con energía todo el tiempo.

Tarjetas Flash

Combinan las mejores características de las SRAM y las ROM/OTP. Al igual que la ROM, los componentes flash no son volátiles. Asimismo, al igual que los chips SRAM, los usuarios pueden escribir, modificar y borrar la información almacenada en ellas.

Funcionamiento de una impresora

La operación de salida tiene lugar cuando pasan de memoria principal a los registros, datos a imprimir, a razón de un byte por vez. Estos bits, a través del bus de sistema llegan a un registro port de datos de la interfaz, que los guarda temporariamente. De este port, cada byte a imprimir pasa en paralelo a la memoria de almacenamiento temporario de este periférico. La electrónica de la impresora se encarga de convertir la información binaria a imprimir en señales graficas de tinta.

Monitor

La plaqueta de video, que se enchufa a la motherboard, además de portar la electrónica intermediaria, contiene una pequeña porción de la memoria principal denominada VRAM, donde se almacena la información a visualizar. Esta llega a la VRAM gracias a la ejecución de un programa por el microprocesador y es convertida en señales analógicas por la electrónica intermedia. Por tres conductores del cable que une la plaqueta de video con el monitor, estas señales llegan a la electrónica del monitor que se encarga de convertirlas en una imagen visible en pantalla.

Unidad IV: Arquitectura.

Esquema general de la CPU

La arquitectura está relacionada con la organización de los registros en la CPU, el número de bits de los buses de dirección y datos, entre otras cosas.

Instrucciones

Cuando la computadora realiza una tarea a pedido del usuario, ejecuta una serie de pasos simples representados por su propio juego de instrucciones, que constituyen su lenguaje de máquina. La computadora solo ejecuta códigos de instrucción en lenguaje de máquina.

<i><u>Instrucciones</u></i> : Unidades elementales que constituyen los programas.

El código de una instrucción es la combinación de bits que la unidad de control de la CPU interpreta para generar las microoperaciones que permitan su ejecución.

La forma de agrupar estos bits en entidades diferenciadas define el formato de instrucción. Una misma unidad de control puede comprender diferentes formatos de instrucción. El más simple comprende un grupo de bits para representar una acción (COP) y otro para representar el dato al que afecta esta acción (DATA). La cantidad de bits en el COP determina el número de acciones distintas que se podrían definir.

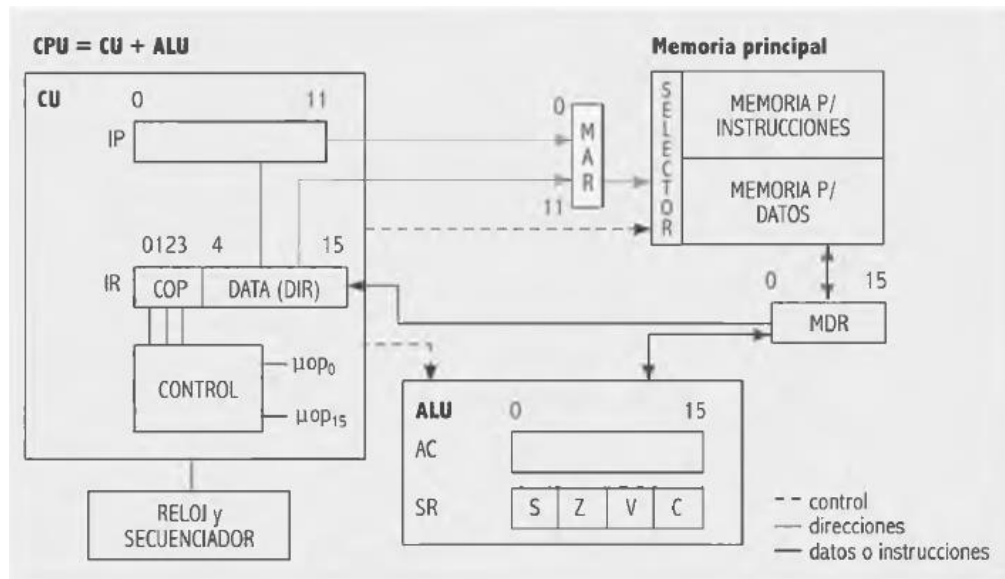


Por lo general el segundo grupo de bits hace referencia a un dato en memoria; determina la posición de memoria donde se aloja el dato.

La unidad de control lee una instrucción, la aloja en un registro interno (Registro de instrucción) e interpreta si el COP afecta a un dato almacenado en memoria, en cuyo caso provoca su lectura. Una instrucción alojada en la unidad de control, está en estado de ejecución y su código binario indica donde está el dato, que acción lo afecta y que módulo de hardware llevará a cabo la acción.

Arquitectura simplificada de una computadora

El modelo de estudio encuadra en los parámetros de una arquitectura Von Neumann



En el centro de todas las operaciones hay una única **CPU** (Unidad de control y procesamiento) que se encarga de buscar las instrucciones de la memoria, interpretarlas y gestionar su ejecución, mientras que los datos son operados en la Unidad Aritmética Lógica (ALU).

La función de la CPU se separa en dos partes:

1. Tratamiento de instrucciones
2. Operación de los datos

Del tratamiento de instrucciones se encarga la unidad de control (CU) y de la operación de datos se encarga la ALU.

La función de la CPU es gestionar el procesamiento de la información. La CU toma las instrucciones de los programas almacenadas en memoria para interpretarlas y ejecutarlas, una por vez, una tras otra, en secuencia. Este proceso se divide en las siguientes etapas:

1. Búsqueda de la instrucción en memoria
2. Interpretación del código de instrucción
3. Búsqueda del dato afectado por la instrucción. (Si afecta a dato)
4. Generación de órdenes al módulo que opera sobre ese dato.

La etapa 1 es la fase de búsqueda o fase fetch, mientras que las otras 3 se agrupan en la fase de ejecución.

Cuando la CU ejecuta cada instrucción de un programa debe alternar entre sus etapas fetch y execute desde la primera hasta la última instrucción. A esta alternancia se le llama ciclo de instrucción, y es el tiempo que tarda la CU (Unidad de control) en cambiar entre la etapa de búsqueda y la de ejecución.

Ciclo de instrucción: Tiempo que tarda la CU en cambiar entre la etapa de búsqueda y la de ejecución

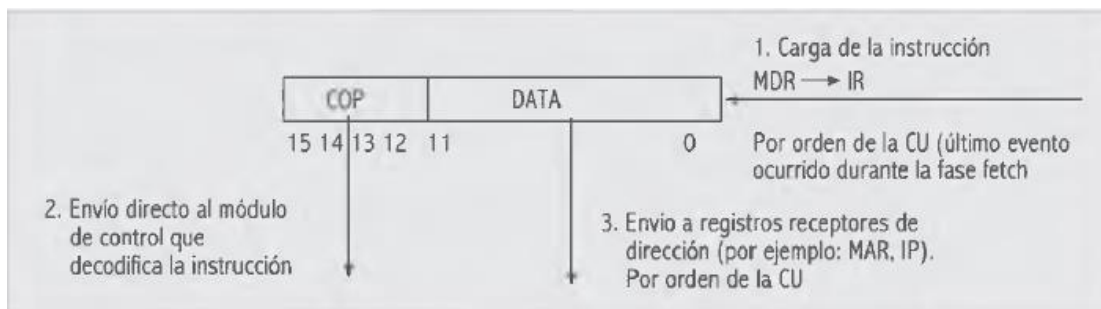
Fase fetch: búsqueda de una instrucción en memoria.

Inicia cuando La CU envía a la memoria la dirección de la palabra donde se encuentra la instrucción, una orden de lectura y una orden de transferencia de la instrucción a la CU. La CU retiene la dirección de la instrucción en un registro denominado IP (instruction pointer o contador de programa) que apunta a la próxima instrucción de programa que se ha de ejecutar. La longitud del IP depende de la cantidad de bits que se necesiten para apuntar a cualquiera de todas las direcciones de memoria posibles. Luego de una ejecución el IP debe incrementarse en una unidad para señalar siempre la próxima instrucción ejecutable.

Una vez que la CU envía a memoria el contenido del IP, da la orden de lectura para que la palabra implicada se almacene en el registro de palabra de memoria. El tamaño de la palabra será la unidad de direccionamiento de la computadora (tamaño de palabra).

La etapa de búsqueda finaliza con la transferencia de la instrucción leída al registro de instrucción, donde permanece almacenada mientras dure su ejecución.

Ese registro se denomina registro de instrucción (IR) y soporta el conjunto de bits del código de instrucción. El IR se relaciona directamente con la CU.



El registro se divide en dos partes: COP y DATA. En este caso el tamaño de palabra es de 16 bits y por lo tanto, lo serán los registros. El COP es de 4 bits y DATA de 12. El registro COP se encarga de interpretar y decodificar la instrucción a realizar, mientras que el sector DATA se relaciona con dos registros receptores de dirección: Registro de direccionamiento de memoria (MAR) y el IP.

El sector DATA se relaciona con el MAR cuando se debe tomar un dato de memoria y con el IP cuando se debe romper la secuencia normal del programa. Esta zona se ignora cuando la instrucción no afecta dato alguno. En una flag, se retiene un 1 durante la fase de búsqueda que

cambia a 0 cuando comienza la fase de ejecución. Cuando esto sucede, la CU actualiza el IP y entra en la fase execute de la instrucción buscada. La función de la flag es importantísima, ya que permite alternar de una fase a la otra, inhibiendo órdenes de búsqueda durante la fase de ejecución y al revés.

Los componentes de hardware implicados en la fase fetch y la secuencia de sus eventos, son los siguientes:

1. IP -----► MAR
2. Word -----► MDR (por orden de lectura)
3. MDR-----► IP
4. IP + 1 -----► P
5. 0 -----► F

Estas microoperaciones definen la fase fetch, y se producen cuando el biestable F está en 1:

- La primera, es una transferencia del contenido del puntero IP al MAR para seleccionar la palabra que se ha de leer.
- La segunda es una orden de lectura a la memoria para hacerla efectiva
- La tercera es una transferencia de la instrucción al IR.
- La cuarta es la actualización del IP Para que señale la próxima instrucción a ejecutar.
- La quinta corresponde a la actualización del biestable F para entrar en fase execute.

Fase execute

Una vez que es efectiva la búsqueda, la CU entra en fase de ejecución. Cada código de instrucción tiene asignada una secuencia de microoperaciones definida en el control. Entre las microoperaciones del código de instrucción se encuentran:

- La transferencia del dato almacenado en palabra a los acumuladores
- Suma binaria
- Desplazamiento de bits
- Almacenar resultados en memoria
- Suspensión de microoperaciones

Unidad de control y sincronización del tiempo.

Cada microoperación debe ser controlada por una variable fundamental, el tiempo. Las señales de tiempo que afectan a las microoperaciones están reguladas según el tiempo de respuesta de los registros que involucran.

Las señales de tiempo son generadas por un sistema de reloj, constituido por un oscilador y circuitos asociados que generan pulsos. Se denomina ciclo de reloj, al intervalo entre dos pulsos consecutivos y se identifica como 1 hercio.



Una computadora controlada por un sistema de reloj se denomina sincrónica, y cuentan con secuencias de tiempo repetitivas formadas por 4,8 o 16 señales de tiempo.

El tiempo de una secuencia constituye el ciclo de la computadora o ciclo de máquina. El cual debería ser compatible con el ciclo de memoria (tiempo de acceso a la memoria), que es el tiempo que tarda la CU en buscar la información en la memoria y dejarla disponible en el MDR.

La primera instrucción del programa siempre se carga en la palabra 000H. Cuando el usuario selecciona el inicio de la ejecución de un programa, se desencadenan una serie de operaciones internas para la búsqueda de la primera instrucción de memoria. A partir de ese momento, la unidad de control estará en uno de dos estados: búsqueda de la instrucción (o fase fetch) o ejecución de la instrucción (o fase execute). La unidad de control actualiza la flag de control de fase para pasar de un estado a otro. Todos estos componentes brindan cierta información al control que genera, considerando estas variables, las “funciones de control” necesarias para llevar a cabo el procesamiento: las microoperaciones.

El módulo de cálculo: unidad Aritmético-lógica

El módulo de tratamiento de datos es la unidad aritmético-lógica (ALU). Aquí los datos se tratan según ordenes de la unidad de control (CU) que interpreta la instrucción durante su estado de ejecución.

Las instrucciones que hacen referencia a la ALU son las correspondientes a los operadores aritméticos suma, incremento, complemento, división y multiplicación por la base binaria y los operadores lógicos AND y XOR. La mayor parte de las instrucciones de una computadora requiere la intervención de la ALU.

La ALU está implementada en torno al registro acumulador. Cuando la operación se realiza sobre un dato, éste se almacena en el acumulador y el resultado de la operación permanece en este registro.

Las operaciones que generan operaciones sobre un solo dato son: puesta a 0 del acumulador, inversión de los bits del acumulador, sumar una unidad al dato cargado al acumulador y desplazamiento de un bit a izquierda o derecha del dato almacenado.

El resto de las operaciones aritméticas o lógicas afectan a dos datos, en consecuencia, el primer dato se almacena en el acumulador y el segundo permanece en el separador de memoria mientras que el resultado final se almacena en el acumulador.

Las banderas del registro de estado asociado a esta unidad, permiten que el programador controle la condición final de una operación para establecer o no una ruptura de la secuencia normal del programa. Cada bandera se actualiza después de una operación en la ALU.

El overflow se produce cuando el resultado excede la capacidad del acumulador. Las instrucciones de salto se provocan cuando la condición de la bandera es verdadera.

Desplazamiento de bits en los registros de la ALU

Los bits se desplazan en un registro por las siguientes razones:

- Para multiplicar
- Para dividir
- Para testear un bit

En sistemas de numeración, el desplazamiento permite multiplicar o dividir un número por su base. En binario, para multiplicar un 2 es necesario agregarle un 0 a la derecha, que es equivalente a desplazar todos sus bits a la izquierda.

Comparación mediante resta

La comparación es función de la ALU. Se efectúa una sustracción sin necesidad de almacenar el resultado y se pregunta por la bandera indicadora de zero o de signo. Entonces si $Z=0$ y $S=1$, el primer operando es menor que el segundo; y si $Z=1$ y $S=0$ El primer operando será mayor que el segundo.

Incremento y decremento de un registro

La operación se realiza sobre un solo operando y, por lo general, afecta al registro acumulador.

Operaciones lógicas

Hay 4 operaciones lógicas que permite realizar la ALU:

- Complementación: Se aplica a un solo operando, invirtiendo el valor de cada uno de los 8 bits. El cálculo de una resta se hace mediante la suma del complemento del sustraendo.
- AND lógico: Se aplica a dos operandos, consiste en hallar el producto lógico (bit a bit) de cada par de bits, que consiste en multiplicar cada bit.
- OR lógico: Se aplica a dos operandos, y consiste en hallar la suma lógica de cada par de bits.
- XOR lógico: Se aplica a dos operandos y permite comparar si ambos son iguales, en cuyo caso el resultado es 0.

Registro de estado asociado a la ALU

Cada vez que la ALU realiza una operación elemental, mantiene cierta información que indica el final de esa operación. Esta información queda retenida en un registro, el registro de estado, donde cada bit se denomina flag e indica con un 1 la condición de verdad y con un 0 la condición de falsedad de un estado.



S es indicador de resultado. Si S=1 el resultado es negativo y si S=0 el resultado es positivo

Z es indicador de cero. Si z=1 el resultado es 0 y si z=0 el resultado no es cero.

O es indicador de overflow. O=1 indica que el resultado supera la capacidad prevista para ese formato y O=0 que no se produjo desborde. Entre otros.

Transferencia de información

Debido a la necesidad de conectividad de los diversos componentes para formar un sistema, se ha establecido una jerarquía de niveles de comunicación, desde las transferencias elementales en la CPU hasta las transferencias completas de bloques de información que ocurren en los buses de entrada/salida.

Sistemas entrada/salida: componentes hardware que se involucran en la interconexión.

Buses

Los bits que representan datos, instrucciones o resultados, pueden existir dentro de una computadora de dos formas:

- Almacenados
- Transmitiéndose por líneas conductoras a velocidades electrónicas desde su lugar de almacenamiento a otro.

Estas líneas electrónicas son los buses.

Bus: elemento de comunicación que relaciona componentes o dispositivos. Son un conjunto de conductores que transfieren señales eléctricas en forma pasiva.

El controlador de bus es un hardware que regula la actividad de los buses, entre sus tareas esta controlar el sentido de la transferencia de datos entre dos unidades. La transmisión de información se hace a través de los buses por los que se transfieren tensiones que representan los 0 y 1.

Los tipos de señales que se transmiten se pueden clasificar según su tipo de información en señales de dirección, control y dato.

Los soportes físicos más comunes son las pistas de circuito impreso y los cables planos. Las señales que representan bits sobre el bus son digitales y pueden transmitir el dato en serie o en paralelo. Cuando el bus está controlado por el reloj del sistema, se realizan transferencias sincrónicas; en cambio, cuando su operación está controlada por los dispositivos conectados a él, realiza transferencias asincrónicas.

Jerarquía de buses

1. Buses internos al chip: Son los más simples, ya que tienen el menor nivel de complejidad. Comunican registros en un chip.
2. Buses que conectan chips sobre una placa: Los distintos módulos que albergan chips se comunican entre sí conectándose a una placa o tarjeta. Estos módulos se interconectan a través de buses. En general estos buses realizan transferencias sincrónicas. Las señales que constituyen el bus de dato son bidireccionales.

Transmisión de bits en paralelo y en serie

- En paralelo: Permite enviar varios bits juntos de un lugar a otro por varios cables.
- En serie: supone enviar por un solo cable uno tras otro, los bits que se quieren transmitir.

Mientras en la transmisión en paralelo con n cables se pueden enviar y recibir n bits simultáneamente, en la transmisión serie enviar bits demanda n veces el tiempo asignado a la duración de cada bit. Es por eso que en el interior de una computadora solo se transmiten bits en paralelo, a fin de enviar millones de bytes por segundos. La transmisión en paralelo solo puede darse entre lugares próximos, debido a que la interferencia electromagnética aumenta con la distancia. Es por eso que una impresora se conecta en paralelo si está próxima a la computadora y en serie en caso de estar a varios metros.

Buses que conectan distintas placas

La conexión entre distintas placas necesita buses que permitan su interconexión. Estas placas están interconectadas por medio de buses. En este nivel el control de la transferencia es más complejo, dado que las diversas placas provienen de distintos fabricantes y se necesita compatibilidad en la forma de transferir un dato.

Buses de entrada/salida

Los buses afectados a la entrada y la salida de la información determinan una multiplicidad de estructuras que se denominan arquitectura de buses. Esta arquitectura permite definir normas de comportamiento para la transferencia de datos desde o hacia los dispositivos de entrada/salida. Su diseño involucra la determinación de controles de estado durante y luego de la transferencia, las velocidades admitidas por los distintos componentes y la cantidad de bits de datos para enviar entre ellos.

DISPOSITIVOS DE ENTRADA/SALIDA (Sistemas E/S):

La función de los periféricos es convertir señales externas que representan datos, en internos en las operaciones de entrada, o a la inversa en las operaciones de salida. En una pc existen periféricos fuera del gabinete y otros dentro del mismo.

En una operación de entrada los datos que llegan del exterior a un periférico tienen como destino la memoria principal, mientras que en una operación de salida el movimiento es el contrario.

Se denominan dispositivos de E/S tanto a las unidades periféricas en sí como aquellas "intermediarias", que se encargan de efectivizar una transferencia entre la memoria interna y la memoria externa en los periféricos.

En toda transferencia se utilizan señales de control, dato y dirección. Las señales de control y tiempo se utilizan para regular la transferencia elemental, indicando como y cuando debe ocurrir. Las funciones de la lógica o inteligencia de un bus E/S son:

- Conectarse con el periférico y el sistema CPU-memoria
- Controlar la temporización durante la transferencia
- Almacenar temporalmente bits para paliar la diferencia de velocidad entre emisor y receptor
- Detectar si se produjeron errores durante la transferencia.

Cuando la actividad del bus E/S es sincrónica se utilizan señales del clock que regulan la transferencia. El bus requiere que los dispositivos conectados a él estén sintonizados a esa frecuencia.

Cuando la actividad del bus es asincrónica, las señales del clock no regulan su operación. Su velocidad depende de los tiempos de los dispositivos conectados a ellos.

Cada dispositivo de E/S tiene asociada una combinación binaria que le permite identificar las señales de dirección para saber de quién viene y hacia donde va.

Las señales de dato representan los bits del mensaje que se ha de transferir. El ancho del bus (Cantidad de líneas afectadas a la transferencia) da una medida potencial de su trabajo; a mayor cantidad de bits transferidos en paralelo mayor capacidad de transferencia.

Las unidades de comunicación con el bus son unidades hardware, que actúan de intermediarias en la comunicación CPU-memoria interna y controlador-periférico.

La elección del tipo de intermediario tiene que ver con las características de cada dispositivo que se ha de conectar.

Los dispositivos E/S permiten la comunicación de la CPU/memoria con el medio externo. O sea que son aquellas unidades que permiten la entrada o la salida o ambas.

La relación entre la CPU y los periféricos no puede ser directa sino que se necesitan nexos físicos que establezcan los controles para lograr una transferencia. Estos también forman parte del conjunto dispositivos de entrada/salida.

CONTROLADORES

Un controlador de periférico es un dispositivo asociado en forma directa al periférico, que puede estar físicamente integrado a él, o bien, separado de este. Está constituido por:

- Un buffer interno (Memoria RAM) que permite el almacenamiento de la información que viaja desde o hacia el soporte
- Una lógica de control, que interpreta comandos de periférico, genera señales para su ejecución y gobierna así la unidad.

Cada dispositivo distinto se asocia con su propio controlador. Su función se puede resumir en, por lo menos, dos operaciones fundamentales.

- Aislar el software de servicio de entrada/salida, que se ocupa de la transferencia de los detalles específicos del hardware del periférico y los convierte en invisibles.
- Compatibilizar la velocidad del periférico respecto de la del resto del sistema.

Cuando un controlador se involucra con una transferencia, la CPU recibe parámetros de un programa que ejecuta; a través de una interfaz, este programa forma parte de la administración de entrada/salida y se denomina manejador de dispositivo.

ADAPTADORES

Un adaptador provee una función para conectar y lograr la operación de un componente conectado a un bus. Puede residir en una tarjeta o en la placa del sistema. Todo dispositivo que no cumpla la función por sí solo necesita su propio adaptador.

PUERTOS DE ENTRADA/SALIDA

Un puerto es un área de almacenamiento alojada en una interface, dedicado a guardar datos en tránsito entre el periférico y la UCP y que permite la comunicación de un periférico con la memoria para enviar o recibir una secuencia de bits.

El software de sistema lo identifica con un nombre.

Puerto serie

Recibe (O transmite), bit por bit, por un solo cable los datos que transmite la electrónica del periférico conectado al mismo. Por la reducida velocidad de transmisión con que operan, el mouse y el modem son los periféricos que típicamente se conectan así.

Puerto paralelo

La transmisión de datos entre ambos subsistemas tiene lugar a través de 8 cables al mismo tiempo, o sea puede realizarse más rápido. Periféricos rápidos, como la impresora están

conectadas de esta manera. Debido a ruidos electromagnéticos, cables en paralelo solo pueden transportar señales hasta unos 4 metros.

Puerto USB

En los 90 las empresas acordaron crear un bus estándar para conectar periféricos de baja velocidad y velocidad media e incluirlo en la placa madre. Entre otras cosas permite:

- Conectar dispositivos mientras el equipo está funcionando, sin reconfigurar el sistema y sin tener que abrir el gabinete
- Facilitar la conexión de dispositivos que operen en tiempo real

El USB no permite transferir directamente datos entre periféricos conectados a él. Cuando se conecta un nuevo dispositivo al USB con la computadora funcionando, el adaptador USB detecta el evento y se produce una interrupción que llama al sistema operativo. La subrutina llamada lee en el dispositivo insertado sus características, el tipo de dispositivo y la velocidad de transmisión que utiliza. Si es compatible con el bus USB le asigna una dirección y ordena escribir esta en los registros de configuración del dispositivo.

El bus del USB inserto en la placa madre está conectado en uno de sus extremos al controlador-adaptador USB/PCI inteligente que permite regular las velocidades de transferencias.

Este bus en su otro extremo tiene por lo menos un conector USB para enchufar un dispositivo de E/S, o un hub USB o algún otro dispositivo. Usando hubs se conectan hasta 127 dispositivos, los cuales funcionan paralelamente y en forma independiente.

El USB no tiene líneas para direcciones o control. El CAUSB decide, cual dispositivo puede enviar o recibir datos a través del conexionado, es el que inicia todas las transferencias. Los hubs tienen incorporados inteligencia para:

- Detectar en sus conectores la conexión/desconexión de dispositivos y configurarlos
- Controlar transferencias de datos y ordenes vinculadas a los dispositivos conectados a él
- Proporcionar tensión de alimentación a dispositivos vinculados al mismo.

Los movimientos de datos que ocurren a través del USB se originan en el software, cuando desde un programa se necesita transferir datos de memoria a un dispositivo periférico o en sentido contrario. Para tal fin se debe ejecutar un software de cliente (CSW) que ve a cada dispositivo conectado al USB como una entidad identificable por un número. Los sistemas operativos cuentan con software para manejar el USB.

En el USB cada transferencia de datos entre MP y un dispositivo consta de una o más transacciones, mediante el envío de paquetes.

Se dan 4 tipos de transferencias, solicitadas mediante paquetes:

- Isócrona: Se transmite una secuencia de paquetes de tamaño fijo a intervalos regulares.
- Masiva: Las transmisiones de los paquetes no son periódicas, pero se efectúan a altas velocidades, a fin de enviar rápidamente importantes cantidades de datos.
- Interrupt data: Dado que el USB no reconoce interrupciones, una subrutina del sistema operativo pregunta cada 50mseg si sucedió una interrupción.
- De control: Transferencias entre el CAUSB y un dispositivo, para leer en los registros de ésta información necesaria para su configuración.

Las características de un dispositivo que interesa determinar y registrar en su configuración automática son:

- identificación del fabricante
- clase de dispositivo
- capacidad de manejo de potencia.

USB 2.0

Las normas 2.0 definen USB con velocidades de transferencia de hasta 60 Mbytes/seg con lo cual pueden conectarse dispositivos de vídeo y de almacenamiento. Se establecen 3 tipos de dispositivos:

- Low Speed (LS): hasta 10.000 B/S (Teclado, mouse, joysticks, realidad virtual)
- Full Speed (FS): de 50.000 B/S a 1MB/S (audio, micrófonos)
- High Speed (HS) de 3 a 50 MB/S (video, almacenamiento)

INTERFACES

Una interfaz es un hardware intermediario ubicado entre dos subsistemas independientes, que sirve para comunicarlos y adaptarlos eléctricamente. En las PC cuando se habla de interfaz se asume que se trata de una plaqueta adaptadora, insertable en la plaqueta principal, cuyos circuitos están conectados a la electrónica de un periférico y por otro lado a la UCP. Dichos circuitos contienen registros de almacenamiento transitorio (ports), donde programas dejan órdenes para el periférico y para retener datos en tránsito entre este último y la UCP.

TRANSFERENCIAS DE ENTRADA/SALIDA:

Los aspectos fundamentales para resolver durante una transferencia son:

- -Sincronizar los tiempos de transferencia entre la CPU-memoria y el periférico.
- -Decodificar los bits que identifican al dispositivo. El dispositivo debe incluir una lógica de direccionamiento que le permita reconocerlo
- -Convertir, si es necesario, el mensaje enviado de un código a otro
- -Controlar que el mensaje enviado se reciba en forma correcta.

Se denominan maestros a los dispositivos que tienen el control del bus en un momento determinado. Un maestro puede enviar las señales de control, dirección y dato sobre el bus. Los dispositivos restantes, conectados al bus pero que no lo controlan, se denominan esclavos: Un

esclavo puede pedir un servicio de transferencia, pero no inicializarla. La estrategia de control del bus depende de la arquitectura diseñada para el sistema y se denomina "arbitraje" del bus.

Todas las responsabilidades descritas se reparten entre la CPU, la interfaz y el controlador periférico.

DRIVERS

Son los programas que "conocen" al dispositivo periférico. Todas las particularidades de los periféricos son conocidas por el sistema operativo por medio de drivers, que son programas en cuya codificación se hace referencia a los comandos propios de cada periférico. Un driver de un periférico es un programa bastante diferente a un driver de otro. Cuando un driver es llamado para producir un servicio, debe conocer que controles y actividades se deben realizar y en qué orden se debe enviar la secuencia de operaciones al controlador del periférico. Además el driver debe contener una serie de instrucciones que permitan la evaluación final de la o las operaciones realizadas y debe poder "armar" un informe de errores.

COMANDOS DE LOS PERIFERICOS

Los periféricos son controlados por configuraciones de bits que hacen referencia a que es lo que el dispositivo debe hacer. Los comandos se pueden clasificar en:

- Comandos de verificación: Son aquellos que permiten evaluar si el periférico esta:
 - Prendido o apagado
 - Ocupado
 - No operable
 - En error de operación
- Comandos de control: Son aquellos que ordenan al periférico:
 - Prenderse o apagarse
 - Saltar de página en una impresora
 - Leer
 - Escribir

Modalidades de entrada/salida

El sistema operativo cuenta con programas que gestionan las transferencias de entrada/salida. Estos programas realizan funciones comunes a todos los dispositivos periféricos, más allá de las particularidades de cada uno de ellos, es independiente de la organización particular de cada unidad.

Los programas de aplicación llaman a los programas de E/S del sistema operativo mediante una llamada al sistema, que puede expresarse de distintas maneras según el sistema operativo que se utilice.

Para llevar a cabo la transferencia completa se pueden identificar modos o métodos:

- Transferencia controlada por programa
- Transferencia iniciada por interrupción
- Transferencia con acceso directo a memoria

Transferencia controlada por programa:

En este tipo de transferencia, la que ejecuta el programa de E/S es la CPU, que controla el acceso a memoria para ubicar el dato, a través de sus propios registros internos. La CPU debe verificar el estado de la interfaz a través del puerto de control en forma continua.

Esta modalidad se conoce como "por sondeo" que significa que es la CPU quien se ocupa de verificar el estado del periférico y no el periférico el que interrumpe la CPU.

Transferencia iniciada por interrupción

La CPU, que es quien solicita la transferencia, no verifica de manera continua el estado de un dato listo para transferirse desde el dispositivo externo, sino que la interfaz asociada genera un aviso que indica que está preparada para transferir, una interrupción. Esto se indica con un bit de estado denominado bit de interrupción.

Entonces la CPU queda en condiciones de ejecutar el servicio de tratamiento de interrupciones que permitirá la transferencia entre el dispositivo y un área de memoria interna. Finalizada la transferencia se reanuda el proceso suspendido.

Interrupciones

Una interrupción supone la suspensión temporaria de la ejecución de un programa, para pasar a ejecutar una subrutina de servicio de interrupción, la cual en general no forma parte del programa. Luego de ejecutarse dicha subrutina, debe reanudarse la ejecución de dicho programa. Por lo tanto, son la forma en las que se llama para su ejecución, a subrutinas de un sistema operativo. Existen interrupciones por software y por hardware.

Interrupción por hardware externa

La interfaz genera una señal de interrupción que se envía por un cable hacia un subsistema ligado a la UCP para solicitarle la interrupción del programa en curso. Esta interrupción tiene su origen en el hardware: La activación de un cable de control del bus, y tiene lugar cuando en el periférico conectado se concretó algo (En el teclado se apretó una tecla, etc.).

La activación de dicha señal causará que en algún momento se interrumpa momentáneamente la ejecución de un programa y que la UC pase a ejecutar la subrutina. Al finalizar la ejecución de dicha subrutina, podrá seguir la ejecución del programa interrumpido. Estos eventos, no pueden ser previstos en un programa cuando sucederán. Por lo que hay un conjunto de líneas de solicitud de interrupción, que sale de cada interfaz y cada línea es distinta.

Todas estas líneas se dirigen a un chip que, en caso de activarse varias solicitudes simultáneamente, da curso a una, las restantes deben esperar. Cuando termina de ejecutarse una instrucción, la UC censa si la línea de control indicadora de solicitud de interrupción está activa. Si el programa en ejecución lo permite, el mismo es interrumpido. Sino dicha solicitud espera hasta ser atendida.

Interrupción por software

Se realiza ejecutando el código de instrucción que ordena llamar a subrutinas del sistema operativo cuando necesita el servicio de una de ellas. A diferencia de las interrupciones por hardware, este tipo de interrupciones tienen establecido en que momento de la ejecución de un programa sucederán. Dado que se trata de una instrucción que está en determinado lugar de un programa. El mecanismo de una interrupción por software:

1. Un programa en ejecución, necesita llamar a una subrutina del sistema operativo, por ejemplo leer un archivo en disco.
2. A tal efecto existe en el programa la instrucción de la interrupción. Puesto que no puede seguir la ejecución de las instrucciones siguientes del programa hasta que no se haya efectuado la subrutina (Leer el disco).
3. Se ejecuta la interrupción y se halla la subrutina
4. Se ejecuta la subrutina
5. Luego de ejecutarse la subrutina correctamente, el SO autoriza la reanudación de la ejecución del programa interrumpido
6. La ejecución del programa se reanuda.

Hay que tener en cuenta que en ambos tipos de interrupciones, las subrutinas no se ponen en ejecución por sí mismos, sino que son llamados por las interrupciones.

Como se retorna al programa interrumpido

En ambos casos de interrupción, antes de ejecutar la subrutina, la UC guarda automaticamente en memoria la dirección de retorno, donde está la instrucción a partir de la cual el programa interrumpido continuará.

Transferencia con acceso directo a memoria:

Las interfaces pueden conectarse a la memoria a través del controlador de acceso directo a memoria (DMA). Los DMA están asociados a dispositivos rápidos y que transfieren la información en bloques, grupos de bytes.

La CPU solo interviene indicándole como parámetros la cantidad de palabras a transferir, la posición inicial de la palabra en memoria interna y la dirección del dispositivo, y queda liberada para realizar otra actividad.

La técnica consiste en relacionar la funcionalidad DMA con el bus que conecta la CPU con la memoria principal o interna, y que ese controlador establezca la relación memoria-interfaz sin intervención de la CPU.

Relación CPU-DMA para activar la transferencia

La CPU se relaciona con el DMA como con cualquier interfaz, le envía a través del bus de direcciones un address que reconocerá como propio y así activará su operación. Además la CPU debe enviar al controlador DMA la dirección de memoria principal donde debe tomar los bits a escribir o donde debe almacenar los bits leídos, la cantidad de palabras de memoria a transferir, una orden de lectura o escritura y un comando que se denomina de inicialización para que el DMA comience la transferencia. Una vez enviada toda esta información, la CPU se desentiende de la E/S, deja el programa que se estaba ejecutando y requería la E/S en suspenso y comienza a procesar otra tarea, si la hay.

Relación CPU-DMA durante la transferencia

El DMA consulta el estado del dispositivo, cuando este se encuentra disponible, le envía al DMA una señal de disponible. El DMA envía una señal que solicita el uso del bus común. Cuando la CPU termina la instrucción en curso atiende la señal y se desconecta del bus, liberándolo y dejándolo a disposición del DMA; luego envía el DMA una señal de bus disponible. A partir de este momento el bus es controlado por el DMA.

Para efectivizar la transferencia del bloque, deja a la CPU inactiva todo el tiempo que dure la transferencia completa, o bien, el período que tarde en efectivizar una transferencia elemental para que la inactividad solo implique un ciclo de memoria.

Relación DMA-dispositivo durante la transferencia:

Cuando el DMA recibe la señal de bus disponible, envía a la memoria principal la dirección de la palabra a transferir a través del bus de direcciones y envía una señal al dispositivo que implique transferencia. En el caso de una salida el DMA envía la señal de lectura a la memoria, que corresponde con la palabra sobre el bus. En el caso de una entrada y tras recibir la señal de transferencia, el dispositivo coloca sobre el bus la palabra a escribir y luego el DMA genera la señal de escritura a la memoria principal. Por cada transferencia elemental el DMA decrementa su registro de cantidad de palabras a transferir en incrementa su registro de dirección de palabra a transferir. Luego verifica la señal disponible del dispositivo; si esta se encuentra activada y la cantidad de palabras a transferir no llega a 0, el DMA inicia la transferencia siguiente. Si opera en modo robo de ciclo inhabilita la señal bus disponible, que posibilita el coloquio DMA-CPU. Cuando el dispositivo envía la señal dispositivo disponible, el DMA vuelve a enviar la señal solicita y se repite el procedimiento.

Fin de transferencia con acceso directo a memoria:

Cuando el registro de cantidad de palabras a transferir llega a cero, el DMA inhabilita la señal solicita y genera una señal de interrupción para la CPU. Cuando la CPU testea la señal interrupción controla el registro de cantidad de palabras transferidas.

Unidad V: El microprocesador

Microcontroladores y microprocesadores

Microcontrolador

Un microcontrolador es una computadora completa con un programa de propósito específico, no configurable ya que es diseñada para suministrar una tarea predeterminada. Pertenece a lo que se denomina arquitectura cerrada.

Microprocesador

Un microprocesador es un chip que, junto con la memoria principal, los buses de sistema, los módulos de E/S y los buses de E/S, constituyen la estructura de una computadora que se integra en una arquitectura abierta, ya que es configurable según necesidad, tanto desde el punto de vista del hardware como del software.

Diferencias entre microprocesador y microcontrolador

Las diferencias básicas entre un Microcontrolador y un Microprocesador están en los componentes de la arquitectura interna y el tipo de arquitectura.

Arquitectura abierta VS arquitectura cerrada

El microprocesador forma una arquitectura abierta porque el computador es configurable por el usuario y puede realizar varias tareas. Con microprocesadores se pueden formar sistemas de uso general.

El microcontrolador forma una arquitectura cerrada porque no es configurable por el usuario (es un computador completo) y realiza una sola tarea debido a que sus prestaciones son limitadas. Con el microcontrolador se pueden formar sistemas embebidos (van dentro del dispositivo al que controlan). Si nos paramos por un momento a pensar, podemos ver microcontroladores por todas partes. Cualquier electrodoméstico que realice varias tareas, como si siguiera varios programas lleva uno dentro. Y hay sistemas que no solo llevan uno. En un coche puede haber más de 150

Arquitectura de microprocesadores

El microprocesador es un circuito integrado formado por transistores. Su arquitectura interna contiene la unidad central de proceso (CPU).

Los microprocesadores son ejemplos de chips muy avanzados que varían en velocidad y capacidad de direccionamiento a memoria, registros, niveles de caché, capacidad de transferencia de datos, etc. Una computadora Von Neumann estaba constituida por componentes básicos: Una memoria, ALU, Unidad de control y registros. En la actualidad un microprocesador tiene más componentes que la CPU Von Neumann.

Casi todos los microprocesadores contienen como mínimo lo siguiente:

- UAL
- Varios registros
- Contador de programa
- Circuitería de decodificación de instrucciones
- Sección de control y temporización
- Cerrojos y buffers de datos
- Líneas de control y buses internos
- Varias entradas y salidas de control

Las funciones de la mayoría de las unidades de un microprocesador son:

- Registro de instrucción: Es un registro que contiene la primer combinación binaria de una instrucción
- Decodificador de instrucciones: Esta unidad interpreta el contenido del registro de instrucción, determina el microprograma exacto que se debe seguir para ejecutar la instrucción completa y dirige adecuadamente la sección de control
- UAL: Realiza las operaciones aritméticas, lógicas y de desplazamiento. Los resultados de esta sección se colocan de nuevo en el acumulador vía el bus interno
- Acumulador: Es un registro de propósito general asociado a las operaciones de la ALU
- Contador de programa: Es un área de almacenamiento de bits que siempre apunta a la siguiente instrucción que se va a ejecutar.
- Unidad de control y temporización: Recibe señales del decodificador de instrucciones para determinar la naturaleza de la instrucción que se va a ejecutar. Luego, las señales de temporización y control son enviadas a todo el microprocesador para coordinar la ejecución de las instrucciones
- Registro de estado: Contiene señalizadores de cero y arrastre
- Reloj interno: Genera una señal de reloj para utilizarla en el interior de la MPU que sirven para sincronizar acciones en el sistema completo.

Algunas de las características principales compartidas por casi todos los microprocesadores:

- Conexiones de alimentación: Los microprocesadores requieren una fuente de alimentación regulada
- Tamaño en bits (Longitud de palabra): Los microprocesadores se clasifican normalmente en unidades de 4, 8, 16, 32 o 64 bits.
- Líneas de datos: Los microprocesadores transfieren datos e instrucciones entre la MPU y memoria vía un bus de datos bidireccional.
- Líneas de dirección: Los microprocesadores utilizan buses de dirección de n líneas, a través de los cuales pueden direccionar 2^n bits de memoria

- Líneas de control: La mayoría de los microprocesadores se caracterizan porque tienen todas o algunas de las siguientes líneas de control:
 - Líneas de reloj
 - Líneas de lectura/escritura
 - Líneas de entrada/salida
 - Líneas de interrupción
 - Líneas de re inicialización
 - Líneas de control del bus
 - Líneas de status del ciclo
- Registros internos
- Modos de direccionamiento: técnica utilizada para buscar el operando deseado durante la ejecución de una instrucción

Longitud de palabra

La palabra del microprocesador está asociada con su capacidad de trabajo simultáneo expresada en bits. Es decir, la cantidad de bits que un microprocesador "procesa" simultáneamente como grupo. El procesamiento involucra la obtención de memoria, su operación y el almacenamiento del resultado de nuevo en memoria. En la actualidad muchos de los microprocesadores son de palabra de 64 bits.

Palabra también puede utilizarse para hablar de estructuras de datos y para expresar la cantidad de bits a los que se puede acceder por vez en memoria (palabra de memoria).

Palabra de memoria: Cantidad de bits a los que se puede acceder por vez.

Capacidad de direccionamiento

Tiene relación con el acceso a líneas que transfieren direcciones a la memoria. Estas líneas - bus de dirección - determinan el potencial de direccionamiento del microprocesador, es decir, cuanta memoria como máximo puede visualizar el microprocesador. Con n bits del bus de direcciones se obtienen 2^n combinaciones distintas. Si el microprocesador aumenta en una la cantidad de líneas, duplica su capacidad de direccionamiento.

Número de instrucciones

La cantidad de instrucciones que un microprocesador puede decodificar y por lo tanto ejecutar, implica su mejor capacidad para hacer cosas distintas.

Cada instrucción tiene un número predeterminado de microoperaciones y si se las agrupa como ladrillos o bloques de construcción de un programa, serán consideradas paquetes de microoperaciones.

Número de registros internos

Es la cantidad de registros con los que cuenta el microprocesador, cuya función es sustentar las necesidades de almacenamiento temporal durante la ejecución.

Al conjunto de registros que pueden actualizarse se lo denomina registros para el programador o registros visibles. Existen registros que se utilizan para la administración del sistema y que son de alcance exclusivo de instrucciones de mayor nivel de privilegio. A estos registros se puede acceder por medio de los programas del sistema operativo y se los denomina registros invisibles.

Registros de uso general

15	8	7	0
AH	AL	Ax Registro acumulador (operación E/S y de cadena)	
BH	BL	Bx Registro base (registro base para direccionamiento)	
CH	CL	Cx Registro contador (para bucles, iteraciones, desplazamientos y rotaciones)	
DH	DL	Dx Registro para datos (almacenado de datos, direcciones de puertos, extensión de Ax en multiplicación y división)	

Todos los registros se pueden utilizar en operaciones de cálculo aritmético o lógico. Es factible emplearlos con instrucciones que los afecten en su totalidad (AX), o que afecten solo la parte alta (AH) o la baja (AL).

Registros punteros

Son registros utilizados para desplazarse dentro de un bloque o zona de memoria. Si el bloque es de código "IP", si es un bloque de pila "SP" y "BP" y si es un bloque de dato "SI"

- IP: Registro puntero de instrucción
- SP: Registro puntero de pila
- BP: Registro base para la pila
- SI: Registro índice fuente
- DI: Registro índice destino

Registro de estado

En este registro se alojan, por nombrar algunas, todas las banderas aritméticas, banderas de modo de trabajo del microprocesador, banderas asociadas a interrupciones, etc.

Registros de segmentos

Son registros concebidos para brindar soporte a aquellos sistemas operativos que administran la memoria como una agrupación de segmentos. Almacenan la referencia binaria a la base de un segmento en memoria. Es decir, bloques de tamaño ajustado al objeto que contienen.

Se denomina objeto al código de un programa, los datos que utiliza y una estructura de datos denominada pila.

Relación entre los registros y el modo de direccionamiento a datos

La menor cantidad de modos de direccionamiento surge de la combinación de acuerdo a la formula siguiente:

Dirección de segmento + dirección base + índice + desplazamiento

La base de un segmento se encuentra en los registros de segmento DS, ES, o SS y se multiplica por 16 antes de sumarse al resto de los argumentos enunciados por la formula.

La base y el índice son valores que se pueden modificar durante el procesamiento. El desplazamiento es una constante que se define en el momento del ensamblado del programa y no puede cambiarse durante su ejecución.

Velocidad del microprocesador

Cada microprocesador tiene su propio reloj interno, cuya frecuencia indica con que velocidad puede procesar bits. Esta velocidad se expresa en Hz, en nuestros días en millones de Hz = MHz o miles de millones de Hz = GHz.

La velocidad no es el único factor que asegura la velocidad de ejecución de programas. Es un parámetro más, por lo que no se debería utilizar como único para comparar micros diferentes.

Función del temporizador

El temporizador es una función que se aloja en el chipset de la placa base. Su función depende del reloj y es mantener una cuenta de tiempo basada en él. Se utiliza para conteo, cálculo de intervalos de pausas o para generar pausas.

Alimentación

La alimentación de los distintos componentes de una computadora proviene de una fuente. Esta es un dispositivo que transforma la corriente que nos entrega la red eléctrica para que sea aceptable para los circuitos electrónicos.

Modos de direccionamiento

Los datos afectados durante la ejecución de una instrucción están alojados en memoria principal, de modo que la instrucción hace referencia durante su ejecución a uno o más bytes que constituyen el dato u operando.

La variación en la técnica para acceder al dato permite una mayor flexibilidad para involucrar estructuras de datos más simples o más complejos.

Es el código de operación de la instrucción el que definirá con bits predeterminados el modo de direccionamiento. Lo que cambia es el código de operación. La forma en que la CPU interpreta esto indicando diferentes técnicas de búsqueda de operandos durante la etapa de decodificación.

Modo de direccionamiento en memoria

El operando reside en la memoria y admite múltiples variantes para expresar la posición donde se encuentra.

Direccionamiento directo de memoria

Cuando el modo de direccionamiento es directo la referencia al dato que se especifica en el campo DATA queda sin alteraciones y el tiempo de captación del dato depende sólo del tiempo de un acceso a memoria.

Direccionamiento implícito

En este modo, el dato queda determinado por el mismo verbo y, en consecuencia, en el código de instrucción en el campo código de operación. Todas las instrucciones que asignen un valor por medio del COP pertenecen a esta categoría. No requieren fase de búsqueda del operando y, por lo tanto, su ciclo de instrucción es más corto.

Modo de direccionamiento inmediato

El operando reside en la propia instrucción pero ahora no en el campo código de operación sino en el campo DATA.

La ventaja es que no se requiere el acceso a memoria para obtener el dato. La desventaja, es que no se puede modificar el valor del operando durante el transcurso de la ejecución del programa ya que es una constante.

Direccionamiento indirecto

El campo DATA contiene una dirección de una posición de memoria que a su vez, contiene la referencia al dato. Para acceder al dato se requieren dos accesos a memoria.

Modo de direccionamiento por registros

El operando reside en un registro interno del procesador. La instrucción hace referencia al número de un registro en lugar de hacerlo a una dirección de memoria.

Considerando que la operación que requiere más tiempo de CPU es la de acceso a memoria, se deduce que cuantos menos accesos a ella se necesiten mayores será la velocidad de procesamiento. Por este motivo, la CPU de las computadoras disponibles en el mercado cuenta

con varios registros internos que se puede inicializar o modificar y cargar con datos de memoria. Las instrucciones pueden hacer referencia a estas memorias locales que son los registros y así optimizar el tiempo de ejecución de los programas.

El código de la instrucción no solo especifica con determinados bits el modo de direccionamiento, sino, con otros bits, el número que identifica al registro que no ha sido cargado con el dato previamente.

Direccionamiento directo por registro

La referencia a un registro siempre es más rápida, puesto que no hay acceso a memoria y por lo tanto se observa una ganancia de tiempo en el acceso al dato.

Direccionamiento relativo a la base

Para acceder a este dato, será necesario conocer la referencia de comienzo en la que se aloja y almacenar dicha referencia en un registro de CPU que se denomina base. Luego se puede acceder, por ejemplo, al quinto elemento sumándole a la base la constante 4

Direccionamiento a una pila

Una pila es una estructura de datos a los que se accede según el criterio LIFO que supone que los datos van ocupando posiciones sucesivas y cuando se quiere leer el contenido de una de ellas solo se puede acceder a la última que se escribió en dicha estructura. El que lleva el control de las direcciones en la pila es el puntero (stack pointer). Su contenido direcciona la primera posición vacía de la pila y se actualiza por cada dato agregado y cada dato extraído.

La pila es un lugar utilizado para almacenar el contenido de los registros de la CPU en forma temporal, lo que permite su reutilización como área de trabajo y la restauración de sus valores originales una vez concluida la operación.

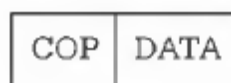
Instrucciones

Programa

Un programa es una secuencia de operaciones o comandos denominados instrucciones, que la computadora ejecuta una a una. La ejecución de una instrucción se ejecuta en 4 pasos básicos:

- 1- Búsqueda de la instrucción en memoria
- 2- Decodificación de la instrucción
- 3-Ejecución de la instrucción
- 4- Actualización del puntero de instrucción para proceder a la ejecución de la próxima.

Cada instrucción se representa con una secuencia binaria, donde cada grupo de bits es interpretado por la CPU como un campo. Cuando se presenta el set de instrucciones que puede ejecutar la CPU, se hace abstracción del contenido binario y se indican que campos componen la



instrucción y como se interpretan. Esto se conoce como formato de instrucción y se puede representar como sigue:

COP: código de operación de la instrucción

DATA: dato a operar.

Formato de instrucción

Manera en la que deben interpretarse los bits que constituyen el código de máquina de la instrucción. En los manuales de cada máquina se especifica el juego de instrucciones que la CPU comprenderá.

Instrucciones sin dirección

La instrucción más simple está constituida por un grupo de bits que deben interpretarse como una unidad. Ésta representa únicamente al código de operación, esto es, como va a ser ejecutada después de haberse leído en memoria y colocada en la unidad de decodificación y secuenciamiento de la CPU

Un ejemplo es la instrucción HLT de 8 bits y que indica a la CPU que debe detener la ejecución de las instrucciones del programa en curso y discontinuar su actividad. Esta instrucción no afecta a datos y su formato se representa solamente con el campo COP (Código de operación.)

Instrucciones de una sola dirección

Todas las instrucciones hacen referencia implícita, en el código de operación, al registro acumulador. Por lo tanto DATA hace referencia al dato afectado.



Dato referido por la instrucción

Una instrucción en la que el dato se debe buscar en la memoria principal, el código de la misma puede hacer referencia a su posición en memoria, en lugar de referirse al dato en sí. Un ejemplo es MOV AX, [FFFF] donde los corchetes indican una información relativa al cálculo de la dirección efectiva donde se aloja el dato. Desde el punto de vista de la ejecución, dos instrucciones "iguales" en términos de funcionamiento pero con distinto código, generan eventos diferentes.

Por ejemplo: MOV AX, 0000 | MOV AX, [FFFF]. En la primera el dato se aloja en forma inmediata al código de operación, por lo que no requiere un acceso a memoria y se mueve desde la instrucción al registro X. En la segunda se debe calcular la dirección efectiva del dato en memoria y luego ordenar su lectura. Una vez transferido el dato vía Data Bus, se lo puede mover al registro AX. Esto implica que el código de operación de la instrucción es distinto; sin embargo, el formato no cambia, es el valor del código el que indica que el contenido del campo DATA1 es una referencia al dato y no el dato en sí.

Instrucciones de dos direcciones

Para reducir aún más el código del programa, algunos diseñadores optaron por agregar un campo DATA2 en algunas instrucciones, de modo tal que se utilice para referenciar al segundo operando. A su vez, referencia el lugar donde se almacena el resultado.



Instrucciones de 3 direcciones

En otro formato se agrega el campo DATA3 para hacer referencia al lugar donde se almacena el resultado.

Instrucciones de 4 direcciones

Este formato utiliza los campos DATA1 y DATA2 para referenciar los datos afectados por el código de operación. El campo DATA3 referencia el resultado y DATA4 referencia la próxima instrucción por efectuarse.

Relación entre el diseño del hardware y la ejecución de instrucciones

La programación en lenguaje de máquina implica el conocimiento del tipo de instrucciones en código de máquina. Para simplificar la tarea de recordar secuencias binarias a la hora de programar, se ha desarrollado un lenguaje simbólico de bajo nivel conocido como Assembler.

Cada computadora tiene su propio Assembler. Su inclusión obedece a la consideración de que la programación de máquina permite comprender de manera más eficaz el módulo más importante de la computadora, el que entiende y genera microoperaciones para su ejecución, llamado unidad de control (CU). Además permite un mejor entendimiento en la interacción entre lo físico y lo lógico, es decir, el hardware y el programa.

Para cada sentencia del programa fuente, corresponderán n instrucciones en lenguaje de máquina.

Tanto el lenguaje simbólico de alto nivel, que es aquel estructurado de una manera más cercana al hombre, como el lenguaje de bajo nivel, más cercano a la computadora; permiten obtener las mismas instrucciones en lenguaje de máquina.

La programación en lenguaje de máquina permite aumentar las prestaciones del sistema operativo. Por lo tanto, debe considerarse satisfactorio el estudio de las instrucciones de máquina que entiende la CPU de su computadora.

Interpretación de un código de máquina

Las instrucciones se implementan en lenguaje de máquina identificando con "campos de bits" los siguientes parámetros:

- Donde se ubica la identificación del registro fuente y el registro destino
- El tamaño de los datos
- El tamaño del desplazamiento
- Los registros de base e índice que se utilizan

Lenguaje maquina

En el nivel más básico, el microprocesador responde a un listado de operaciones que se denomina programa máquina. Este listado de operaciones comprende un programa almacenado en memoria, que el micro debe ejecutar en el orden en que se encuentran en memoria.

Traductores de lenguaje

Son programas cuya función es convertir los programas escritos por el usuario en lenguaje simbólico a lenguaje de máquina. Un programa escrito en lenguaje simbólico se denomina fuente, mientras que un programa en lenguaje de maquina se denomina ejecutable. Las instrucciones del programa fuente constituyen la entrada al traductor de lenguaje, las salidas serán las instrucciones del programa ejecutable. La traducción comprende el análisis del léxico y la sintaxis de cada instrucción o sentencia, así como las referencias lógicas a las que apunta cada instrucción.

Se necesita un traductor para cada lenguaje simbólico que se utilice. Se destacan tres tipos de traductores de lenguaje: ensambladores, intérpretes y compiladores.

Ensambladores

Para transformar la descripción del lenguaje humana del programa a código máquina, se estableció un sencillo lenguaje de programación que utiliza palabras y frases para representar los códigos máquina del microprocesador. Se encarga de traducir un archivo fuente escrito en un lenguaje Assembler a un archivo cuyas instrucciones estén en código máquina, que es ejecutable directamente por el procesador para el que se creó.

Generalmente una frase en lenguaje ensamblador constará de 1 a 3 bytes de código máquina y está constituida por un único nemotécnico de 3 letras.

La programación en lenguaje ensamblador es un método de «humanizar» las operaciones de un microprocesador, por medio de un lenguaje simbólico, que permite la escritura de programas a nivel código de máquina (código binario decodificable por el procesador pero imposible de escribir en la práctica)

Todo procesador, grande o pequeño posee su lenguaje máquina y, por lo tanto, su simbólico de máquina.

Todo lenguaje tiene un conjunto de reglas que el programador debe respetar. Un programa Assembler está organizado con una secuencia lógica de líneas de programa. Se acostumbra a dividir las sentencias del lenguaje ensamblador en 4 campos: Etiqueta, nemotécnico, operando y comentarios

- **Etiqueta:** No se utiliza siempre
- **Nemotécnico:** Contiene el nemotécnico exacto que proporciona el fabricante. Este campo indica al programa ensamblador esencialmente la operación que se va a realizar.
- **Operando:** Contiene información sobre registros, datos o direcciones asociados con la operación.
- **Comentario:** no lo evalúa el ensamblador, solo lo imprime. Es de ayuda para comprender que ocurre en el programa

Con la información de los campos nemotécnico y operando, el programa ensamblador puede generar el código correcto en lenguaje máquina.

Cuando el programa es complejo suele dividirse en módulos que se programan en forma independiente y al momento de traducción pueden estar alojados en zonas diferentes de memoria, que se ensamblan y se ponen a punto de manera independiente del proceso de traducción. Es necesario juntar los módulos de manera de organizar un solo programa ejecutable, donde las instrucciones se ejecuten una detrás de otra.

Interpretes

Es un traductor de lenguaje que traduce una instrucción en lenguaje de alto nivel a lenguaje de máquina y, de ser correcta, la ejecuta inmediatamente. Si se encuentra un error de sintaxis lo señala e interrumpe la ejecución. La ventaja es que el programa se va probando a medida que se confecciona. Su desventaja es que debe traducirse cada vez que se quiere ejecutar.

Compiladores

Es un traductor de lenguaje que traduce un programa escrito en lenguaje de alto nivel a lenguaje de máquina. Separa la traducción de la ejecución del programa y agiliza tanto una como otra. La ejecución del programa solo se realiza cuando la compilación terminó de manera satisfactoria. Permite obtener el código de máquina del programa compilado y hacer un resguardo del mismo en una memoria externa. Luego se puede convocar al programa ejecutable tantas veces como sea necesario, con la única condición de que se lo transfiera desde la memoria externa hacia la memoria principal. La compilación también proporciona un informe del programa fuente con los errores.

El proceso de compilación de un programa puede interpretarse en tres etapas:

- **Análisis del léxico:** se separa la cadena de caracteres en símbolos elementales
- **Análisis sintáctico:** se determina la estructura sintáctica de acuerdo con un patrón de reglas gramaticales correspondiente al lenguaje que se utilice, y se analiza la secuencia de señales generadas en el proceso de análisis de léxico.

- Generación de código: se genera el código de las instrucciones para cada elemento sintáctico del programa.

Una cuarta etapa podría ser la optimización, cuyo objetivo es reducir el programa o hacerlo más veloz, utilizando técnicas como detección y eliminación de instrucciones redundantes y uso de registros asociados a la CPU en lugar de palabras de memoria.

Resumen de instrucciones

El repertorio de instrucciones del microprocesador genérico, disponibles al programador, está dividido en 7 categorías:

1. Instrucciones aritméticas
2. Instrucciones lógicas
3. Instrucciones de transferencia de datos
4. Instrucciones de bifurcación
5. Instrucciones de llamada a subrutina
6. Instrucciones de vuelta o de retorno
7. Instrucciones misceláneas

Operaciones aritméticas:

Son las operaciones de sumar, restar, incrementar, decrementar y comparar.

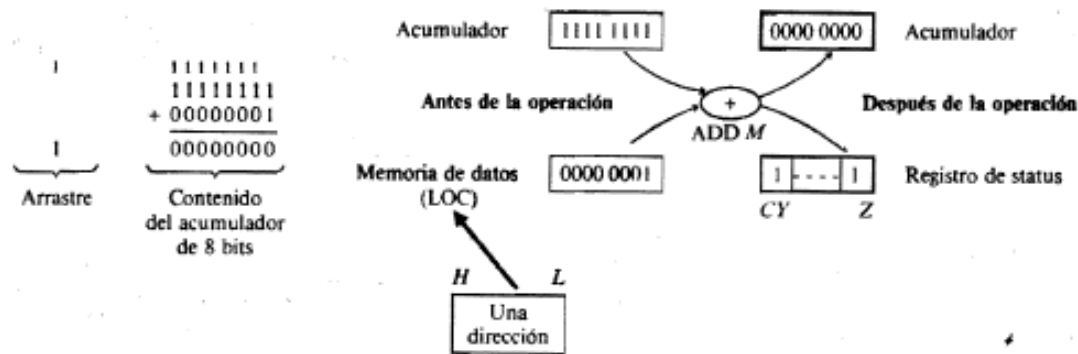
Suma

Hay 4 instrucciones de sumar. El acumulador, identificado como el registro A en el cuadro, contiene uno de los números que se va a sumar. Cada instrucción de suma especifica una fuente diferente para el otro número que va a ser sumado.

1. Sumar A inmediato (ADI): Es una instrucción de 2 bytes. El código de operación aparece en el primer byte de la memoria de programa y el dato a ser sumado al acumulador en el segundo byte. El dato inmediato de la memoria de programa se suma al contenido del acumulador. La suma es devuelta al acumulador después de la operación.
2. Sumar el registro L a A (nemotécnico ADD L): El contenido del acumulador se suma al contenido del registro L. La suma es devuelta al acumulador después de la operación.
3. Sumar H a A (ADD H): Instrucción de un solo byte de suma registro a registro. El contenido del registro A se suma al contenido del registro H. La suma se deposita en el acumulador una vez culminada la operación.
4. Sumar registro indirecto (ADD M): Esta es una instrucción de un solo byte. La posición del dato que se va a sumar es algo más complicada utilizando este modo de direccionamiento indirecto de registro. El registro HL contiene una dirección de 15 bits que señala a la posición adecuada de la memoria de datos. El contenido de esta posición se suma al contenido del acumulador. La suma es devuelta al acumulador después de la operación de suma. Las instrucciones de registro indirecto utilizan un registro de 16 bits como puntero de dirección.

Efecto de una instrucción ADD sobre los señalizadores y el acumulador

Para dar un ejemplo de cómo los señalizadores son afectados por una operación, considerar la suma binaria de 11111111 con 00000001. Como se muestra en el cuadro.



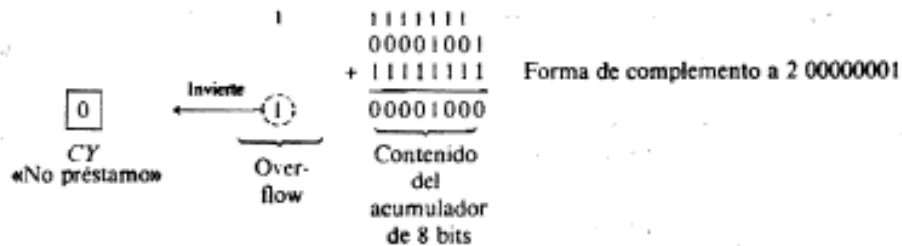
El problema se resuelve por el microprocesador utilizando la instrucción ADD M. El acumulador contiene 11111111 y la posición en la memoria de datos señalada por el registro HL tiene el otro número que se va a sumar 00000001. Cuando se completa la suma, el acumulador contiene los 8 bits menos significativos de la suma (00000000). El señalizador de arrastre del registro de status se pone a 1 indicando que hubo arrastre del bit más significativo del acumulador. El señalizador de cero examina el contenido del acumulador después de la operación y encuentra 00000000. Al encontrar cero, el señalizador de cero se pone a 1 indicando que el acumulador es 00000000 después de la suma.

Resta

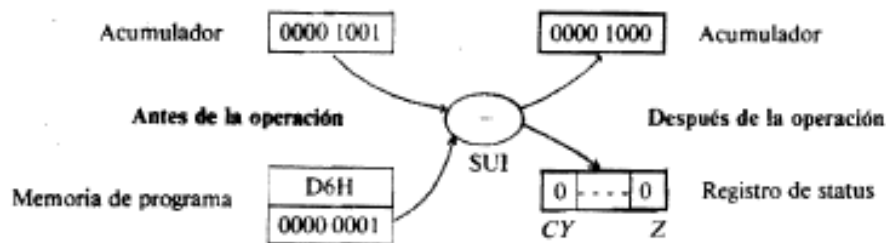
Para las operaciones de resta del microprocesador, cada instrucción resta el contenido de algunos registros o posiciones de memoria del contenido del acumulador. Internamente la ALU del micro no tiene hardware para restar, por lo tanto realiza la sustracción convirtiendo el sustraendo a su forma de complemento a 2 y sumando.

- **Resta inmediata (SUI):** Utilizada en el ejemplo. El dato inmediato del segundo byte de la memoria del programa es restado del contenido del acumulador. La diferencia es transferida al acumulador después de la resta. El señalizador de arrastre se pone a 0 indicando que no ha ocurrido ningún préstamo o que el número del acumulador antes de la operación era mayor que el número de la memoria. El señalizador de cero examina el contenido del acumulador después de la operación de resta. Si el contenido del acumulador es distinto de cero, el señalizador se pone a 0.

$$\begin{array}{rcl}
 00000001 & \xrightarrow{\text{complementa}} & 11111110 \text{ complemento a 1} \\
 11111110 + 1 & = & 11111111 \text{ complemento a 2}
 \end{array}$$



(a) Resta binaria convirtiendo el sustraendo a su forma de complemento a 2 y sumando



(b) Instrucción de resta inmediata

Como ejemplo, el binario 00000001 se resta de 00001001. El sustraendo en este problema es 00001001 que se convierte a su forma en complemento a 2. La forma en complemento a 2 del sustraendo (11111111) se suma al minuendo, dando una suma de 1 0000 1000. El 1 de la posición más significativa es un overflow y no es parte de la diferencia 00001000. El micro utiliza el overflow para afectar el señalizador de arrastre. Al restar, la CPU invierte el overflow y este se convierte en el contenido del señalizador de arrastre (CY). En el ejemplo, el overflow de 1 se invierte poniendo a 0 el señalizador de arrastre, lo que significa que no se ha presentado ningún préstamo o que el minuendo es mayor que el sustraendo.

Comparación

El repertorio de instrucciones del micro genérico tiene cuatro instrucciones de comparación. Estas instrucciones restan el contenido del registro o memoria del contenido del acumulador pero no cambian sus contenidos. Los señalizadores son afectados por las instrucciones de comparación.

- Comparar el registro L (CMP): Números en el acumulador y el registro L son comparados por el micro. Ni el acumulador ni el registro L cambian los contenidos después de la operación de comparación. Sin embargo son afectados los señalizadores del registro de estado. Una vez efectuada la resta, las 8 posiciones

menos significativas son evaluadas para ver si son cero. Si lo son, el señalizador de 0 se pone a 1. El overflow se invierte, por lo que si es 1 se cambia a 0, lo que significa que el contenido del acumulador es mayor o igual que el contenido del registro L

Ver anexo. Tabla instrucciones aritméticas.

Operaciones lógicas

Incluyen las instrucciones AND, OR, XOR, NOT (complementación) e instrucciones de desplazamiento circular. El foco de la mayor parte de las operaciones está en el contenido del acumulador. El modo de direccionamiento indica cómo y dónde se encuentra el otro dato.

AND

- AND inmediato (ANI): El contenido del acumulador realiza la operación AND con el segundo byte de datos de la memoria de un programa. La operación AND se realiza bit a bit y su resultado se coloca en el acumulador. El señalizador de arrastre siempre es cero. Si es cero el resultado de la operación, el señalizador de cero se pone en 1 y viceversa.
- AND registro indirecto (ANA M): El contenido del acumulador realiza la operación AND con el contenido de la posición de la memoria de datos señalada por el registro. Al realizar la operación, el resultado se almacena en el acumulador, el señalizador de arrastre se pone a 0. Se comprueba el contenido del acumulador y si es 0 se pone a 1 el señalizador de 0.

OR

Las cuatro instrucciones OR realizan la operación OR del contenido del acumulador con los de otros registros o posiciones de memoria. Un ejemplo con el contenido del acumulador siendo 11001100 mientras que el registro L contiene 00001111. El resultado de la operación es 11001111. Esta operación se realiza bit a bit.

Desplazamiento circular y arrastre

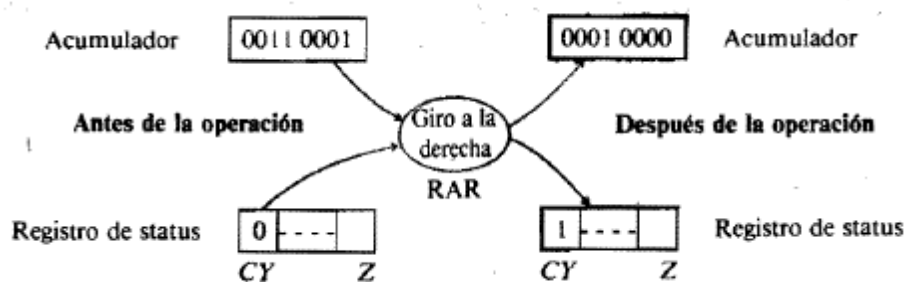
Las dos últimas operaciones lógicas son las de movimiento circular a través del arrastre

Con una o más instrucciones de desplazamiento circular, puede comprobarse la posición de cualquier bit y el señalizador de arrastre puede ponerse a 1 o a 0; pudiendo entonces ser examinado por una instrucción de bifurcación condicional. Otro uso de las instrucciones de movimiento circular es para comprobar la paridad.

Estas instrucciones operan solamente en el acumulador y no necesitan ningún operando de ningún registro o posición de memoria. Por esta razón el modo de direccionamiento se denomina inherente (A veces no se cita ningún modo de direccionamiento para estas instrucciones)

En resumen, las instrucciones lógicas se utilizan para realizar manipulaciones del álgebra booleana en las variables. También se utilizan para poner a 1 ó 0 bits específicos de una palabra y para comprobar y comparar bits

El ejemplo de la figura donde se utiliza la instrucción de desplazamiento circular a la derecha del acumulador a través del arrastre. El contenido del acumulador (00110001) se desplaza una posición a la derecha transfiriéndose el bit menos significativo a la posición del bit señalizador de arrastre. El arrastre actual entrará en la posición del bit más significativo del acumulador. El acumulador contiene 00011000 después



Ver anexo. Tabla instrucciones lógicas.

Operaciones de transferencia de datos:

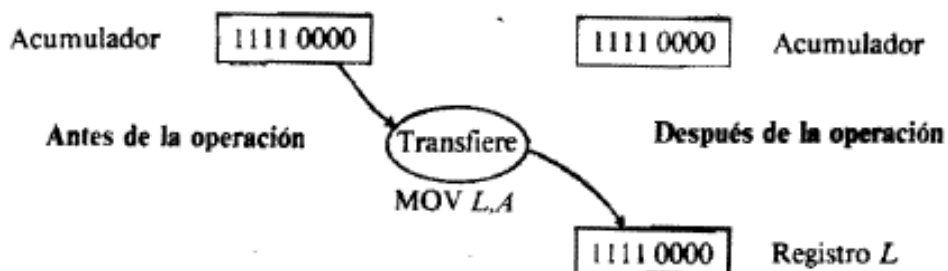
Entre ellas se incluyen las instrucciones de transferencia de registro a registro, de carga de memoria, de almacenamiento en memoria, de entrada y salida y de poner a 1 el señalizador de arrastre.

Casi ninguna instrucción de transferencia de datos afecta a los señalizadores de la MPU.

Cada operación de transferencia tendrá una fuente y un destino para el dato. El modo de direccionamiento determina cómo y dónde se encuentra el dato fuente.

- Transferir del registro o acumulador (MOV): La instrucción MOV(A,L) sugiere primero la acción, que es transferir; la siguiente letra es el destino del dato, mientras que la última letra es la fuente del dato. Este es el convenio utilizado por Intel. Algunos micros identifican la fuente y destino de forma diferente.

Un ejemplo de esta operación es la de la figura. La fuente del dato es el registro A (acumulador),



mientras que el destino es el registro L. El contenido del acumulador es transferido al registro L sin ninguna modificación. El acumulador también contiene este dato una vez completada la operación.

Carga inmediata

Hay 5 instrucciones de carga inmediata. Estas son utilizadas muchas veces para colocar un valor inicial al registro microprocesador antes que el programa.

- Carga inmediata de registro: Un registro se carga con un dato inmediato de la memoria del programa. Esta operación requiere de una instrucción de 3 bytes. El primer byte es el código de operación, mientras que los 2 siguientes son bytes de datos; El segundo contiene el dato de orden superior que se carga en el registro L y el tercer byte contiene el dato de orden superior que se carga en el registro H. Ver ejemplo.
- Carga directa de registro: Esta instrucción utiliza el modo de direccionamiento directo y la operación requiere 3 bytes. El segundo y tercer byte de la memoria del programa son la dirección de 16 bits de la posición de la memoria de datos donde se encuentra el dato que se va a cargar.

Almacenamiento

El micro genérico tiene 5 instrucciones de almacenamiento. Estas operaciones son muy utilizadas para depositar resultados y otros datos en memoria

Ver anexo. Tabla instrucciones de transferencia de datos.

Operaciones de bifurcación

Estas operaciones incluyen a las de salto y bifurcación. Pueden ser denominadas a su vez operaciones de transferencia de control. Estas instrucciones son valiosas debido a su característica de toma de decisiones. Se utilizan para formar ciclos y saltos en el programa.

El micro normalmente ejecuta instrucciones en orden secuencial. El contador de programa siempre sigue la pista de la dirección de la siguiente instrucción que se va a buscar en memoria y ejecutar; generalmente sigue una secuencia ascendente aumentando de uno en uno.

Las instrucciones de salto o bifurcación proporcionan un método para cambiar el valor del contador de programa, alterando la secuencia normal de ejecución del programa.

Salto incondicional o inmediato

Es una instrucción de 3 bytes utilizada para cargar una dirección específica en el contador del programa de la CPU. La información de la dirección sigue inmediatamente al código de operación; y por tanto, se denomina direccionamiento inmediato.

Una instrucción de este tipo se utiliza en un principio para inicializar el contador de programa cuando empieza la ejecución de un nuevo programa.

Estas instrucciones pueden considerarse como un método de cargar una nueva dirección en el contador de programa.

Salto condicional

Estas instrucciones harán que la dirección inmediata se coloque en el contador de programa solo si se satisface la condición especificada. En cualquier otro caso el contador simplemente contará en secuencia ascendente. Un ejemplo es la instrucción salta si cero. Aquí el contador de programa contiene una dirección antes de la operación y continuará en secuencia ascendente hasta que el señalizador de cero se ponga a 1. La CPU evalúa la condición del señalizador de cero en el registro de status, si encuentra que está a 1 (lo que significa que el resultado de la última operación lógica o aritmética fue cero) las condiciones para un salto son correctas y la MPU carga el contador de programa con una nueva dirección. La siguiente instrucción que se ejecutará será la determinada por la memoria de programa. Los saltos o bifurcaciones pueden hacerse hacia atrás o hacia adelante en la memoria del programa.

Ver anexo. Tabla instrucciones de bifurcación.

Operaciones de llamada y vuelta a subrutina:

El microprocesador genérico tiene solamente dos instrucciones en este grupo. Las instrucciones de llamada y vuelta son utilizadas siempre en parejas.

La instrucción de llamada de 3 bytes se utiliza en el programa principal para hacer que el microprocesador salte a una subrutina. La dirección de la instrucción que sigue a la de llamada se guarda en la pila. Las instrucciones de subrutina se ejecutan hasta que la CPU encuentra la instrucción de vuelta. La dirección guardada en la pila es devuelta al contador de programa y la MPU continúa el programa principal donde lo dejó cuando se llamó por primera vez a la subrutina. Se puede utilizar muchas veces una subrutina en la ejecución de un programa principal. La subrutina puede almacenarse en una RAM o ROM.

Ver en anexo. Tabla de instrucciones de llamada y vuelta a subrutina.

Operaciones misceláneas

Incluyen las operaciones de introducir, sacar, no operación y alto.

Las operaciones de introducir y sacar colocan y toman datos de la pila de la microcomputadora y se utilizan por parejas porque lo que se hace en la pila con una instrucción de introducir tiene que deshacerse con una instrucción de sacar. Son muy utilizadas en las subrutinas.

La instrucción no operación no hace nada, excepto consumir 1 o 2 nanosegundos. Es una instrucción de 1 byte que solamente hace que se incremente el contador de programa. No afecta a

los demás registros. Se utiliza como relleno cuando durante la depuración se eliminan una o dos sentencias del programa. Sirven de puente entre dos secciones del programa para que la MPU pueda avanzar de un segmento a otro. También pueden utilizarse para consumir tiempo en un ciclo.

La instrucción alto se utiliza al final del programa para que el microprocesador deje de procesar más instrucciones en la memoria de programa. Solamente una reinicialización o una entrada de petición de interrupción a la MPU pueden arrancar al microprocesador genérico cuando esté en modo de alto.

Ver en anexo. Tabla de instrucciones de llamada y vuelta a subrutina.

Pipeline / paralelismo a nivel instrucción.

Técnica que consiste en dividir el procesamiento de cada instrucción en etapas y que estas operen en paralelo. Es una alternativa para mejorar el rendimiento mediante la ejecución de instrucciones en paralelo.

Mientras se decodifica una instrucción todavía se está ejecutando la anterior, es decir, más de una instrucción está en una misma fase pero en diferentes instancias de ella.

En un modelo que no fuese segmentado, cada instrucción debería estar ejecutada por completo para pasar a la búsqueda de la siguiente. Por lo tanto se deduce que a mayor cantidad de unidades funcionales que puedan operar distintas etapas del ciclo de ejecución de la misma instrucción, más probabilidad de solapamiento y, en consecuencia, un mayor grado de paralelismo. En una situación ideal, la tendencia es lograr la ejecución de una instrucción por ciclo de reloj.

Si bien no se reduce el tiempo de instrucción de una instrucción, en cada pulso reloj se estará ejecutando una etapa de varias instrucciones distintas, lo cual permite ejecutar varias veces más rápido las instrucciones de un programa que en un modelo sin pipe line.

Extraer una instrucción de memoria es lo que más tiempo tarda en llevarse a cabo, ya que depende del tiempo de acceso a memoria. El objetivo es extraerla, decodificarla y redirigirla a unidades funcionales de la EU no comprometidas en la ejecución de la anterior.

Este procedimiento es casi óptimo hasta que aparece una instrucción de salto que rompe la

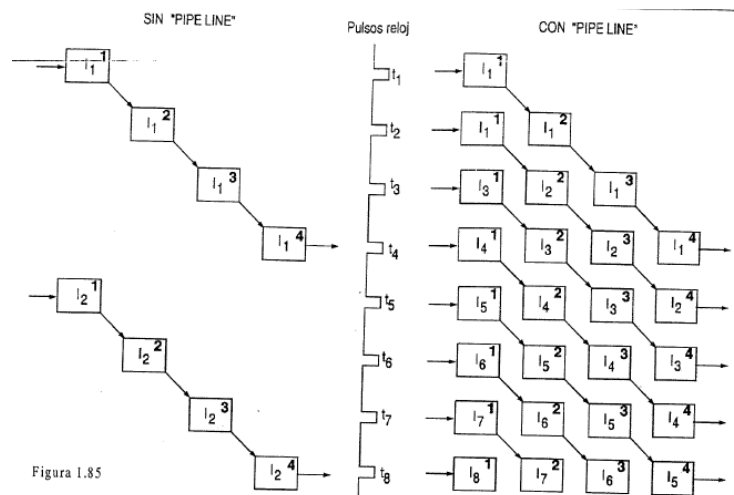


Figura 1.85

secuencia normal de ejecución. Esto implica que la próxima instrucción que se ha de ejecutar no es la siguiente sino que hay que buscar aquella cuya referencia a memoria suele estar indicada en la propia instrucción de salto.

Plataformas CISC y RISC

CISC

Hasta finales de la década de los 70 se pretendía reducir el coste del “hardware”, mediante el uso de complejos juegos de instrucciones basados en la microprogramación. En esta época el tiempo de acceso a Memoria Principal era muy superior al de decodificación y procesamiento de las instrucciones, debido al uso de los circuitos integrados. Para paliar ese desequilibrio se disminuyó el empleo de la Memoria Principal y se aumentó el del procesador, creando así los juegos de instrucciones complejos o CISC.

Las siglas CISC (Complex Instruction Set Computer) hacen referencia a un computador con un repertorio complejo de instrucciones y una UC basada en una ROM de control.

La palabra complejo alude a un repertorio de entre 200 y 300 instrucciones, que según la cantidad de pasos para ser ejecutados pueden ser simples, menos simples y complejas. Por lo que su etapa de decodificación no puede ser muy rápida por requerir una circuitería complicada. Los procesadores CISC tienen pocos registros, lo cual implica más pasos en la ejecución.

En esencia, el procesador puede ejecutar directamente como instrucciones de máquina ciertas sentencias de alto nivel, sin que el compilador necesite traducirlas a instrucciones de máquina, con lo cual el compilador es más simple de construir. En realidad, la traducción la realiza el hardware, dado que la secuencia de pasos simples para ejecutar cualquier instrucción, está memorizada en la ROM de control que forma parte de la UC, donde están guardados dichos pasos. Los pasos de una ejecución están representados por una línea de código binario, que se presenta como salida de la unidad de control por cada pulso de reloj.

Una de las características de los procesadores CISC es que deben contener necesariamente una ROM con los microcódigos para poder ejecutar las instrucciones complejas.

De esta forma, en principio podría ser más rápido obtener dicha secuencia de pasos a realizar de dicha ROM inserta en la UCP, que pedir una secuencia equivalente de instrucciones de memoria principal. Esta ventaja se desvaneció con el advenimiento masivo de las memorias caché y de los pipe line con pre búsqueda de las instrucciones a ejecutar.

Otro de los motivos para usar instrucciones complejas, fue que por ocupar menos espacio de memoria que varias simples que realizan la misma función, podría lograrse ahorrar espacio, en una época en que la memoria era un recurso caro.

En síntesis, una CISC tiene limitaciones en su productividad (Instrucciones/segundos) por requerir una decodificación compleja que insume tiempo; y por ser su pipe line ineficiente si entran en él instrucciones simples y complejas (dado que necesitan distinta cantidad de pulsos para ser ejecutadas). Esto atenta contra el rendimiento "ideal" del procesamiento pipeline, que en términos óptimos supone que todas las instrucciones ocupen prácticamente la misma cantidad de ciclos de reloj para lograr mayor eficiencia.

RISC

La idea de concebir un procesador RISC (Reduced Instruction Set Computer) es que cada instrucción tenga la mayor cantidad de microoperaciones solapadas posibles, de modo que la mayoría de ellas se ejecute en un ciclo de reloj. A su vez, en una arquitectura RISC todas las instrucciones tienen el mismo tamaño, lo que facilita el funcionamiento del pipeline

En los microprocesadores RISC todas las funciones y el control están "cableados" (hardwired), para lograr velocidad y eficiencia máximas. Por lo tanto, se disminuye el tiempo de ejecución, también beneficiado por no acceder a la ROM de control.

Hay sólo dos instrucciones para acceder a memoria, Load y Store, que se utilizan para cargar la gran cantidad de registros disponibles en esta arquitectura para almacenar operandos. Las instrucciones que realizan operaciones aritméticas son de referencia a registro y, en general, tienen especificados dos para los operandos y uno para el resultado. Esta característica permite una independencia entre las instrucciones de acceso a memoria y las de cálculo, lo que facilita la concurrencia de ambas actividades y favorece el procesamiento segmentado.

Al tener instrucciones de tres operandos, el resultado no afecta a ninguno de éstos; en consecuencia, de ser necesaria la reutilización del operando en la ejecución de próximas instrucciones (y si el registro no fue actualizado) ya está disponible en el registro sin necesidad de acceder a memoria de nuevo.

Todas las instrucciones de una plataforma RISC tienen el mismo tamaño para que cada paso en la ejecución de una instrucción sea lo más rápido posible. La mayoría de ellas se ejecuta en un ciclo reloj. Este tipo de plataformas tienen muchos registros internos lo que implica menos accesos a memoria, con la consiguiente ganancia de tiempo.

Los procesadores RISC (Reduced Instruction Set Computer) fueron desarrollados luego de analizar las limitaciones en productividad que presentaban los CISC.

¿Cuáles son las diferencias entre las plataformas RISC y CISC?

Buscando optimizar la performance de los procesadores, se realizaron estadísticas de las instrucciones de máquina más usadas. Resultó que las instrucciones más simples (El 20% del

repertorio de las instrucciones de un CISC) constituían el 80% de programas típicos ejecutados. También se constató que la mayoría de las instrucciones que traducía el compilador eran simples.

Se determinó que en los programas predominaban las sentencias de asignación, los if, los call y los loops. Las llamadas a procedimiento son las que emplean más tiempo de máquina y los ciclos repetitivos, los que consumen la mayor parte del tiempo de ejecución de los programas.

Se verificó que el rendimiento de una arquitectura depende fundamentalmente de la existencia de compiladores inteligentes, capaces de generar código de máquina que optimice asignaciones y loops, y con algoritmos para generar secuencias que ayuden a la productividad del pipeline.

Por eso los compiladores procuran que los datos más utilizados en un lapso de tiempo estén en registros, minimizando los accesos a memoria. Esto implica que un RISC debe tener un gran número de registros. Mayor cantidad de registros implica menos accesos a memoria, con la consiguiente ganancia de tiempo.

Lo determinante de un RISC es que el repertorio de instrucciones es sencillo: todas son de igual tamaño, con el objetivo de simplificar el decodificador y la unidad de control para que cada paso en la ejecución de una instrucción sea lo más rápido posible. Un RISC consta de instrucciones cuya fase de ejecución requiere mayormente un paso o a lo sumo dos.

Las RISC no requieren una ROM de control para guardar cada microcódigo a los fines de comandar el procesador. Esto permite aprovechar el área que ocupa esta ROM en el chip, que se usa para registros y pipelines.

Debido a que no existen instrucciones complejas y que la mayoría de las instrucciones son sencillas (ejecutables en igual cantidad de pasos) aumenta considerablemente el rendimiento de un pipeline RISC en relación al de un CISC.

Arquitecturas típicas

Casi todos los microprocesadores contienen como mínimo lo siguiente:

- UAL
- Varios registros
- Contador de programa
- Circuitería de decodificación de instrucciones
- Sección de control y temporización
- Cerrojos y buffers de datos
- Líneas de control y buses internos
- Varias entradas y salidas de control

Además, Una pastilla microprocesadora puede contener también algunas de las unidades funcionales siguientes:

- Memoria RAM
- Memoria ROM
- Puertos de entrada/salida
- Circuitería de reloj interna

Familia

Una familia de microprocesadores es un conjunto de modelos ligados por algunas características comunes.

Familia X86

Los procesadores de esta familia son del fabricante Intel, y su arquitectura responde al nombre IA-32.

El juego de instrucciones de esta arquitectura es de tipo CISC (repertorio amplio de instrucciones), con lo que cada instrucción es de tipo complejo y se ejecuta en varios ciclos de reloj.

La razón por la cual se estudian estos procesadores es porque son los más usados actualmente y parece que lo van a seguir siendo en el futuro ya que la de Intel es una arquitectura progresiva y moderna.

Generaciones

La familia x86 está compuesta por seis generaciones. Dentro de cada generación hay diferentes modelos que varían en la relación precio/prestaciones y el consumo. Cada modelo está orientado a cubrir el sector de mercado donde ese factor sea crítico. Todos son compatibles entre sí.

Primera generación: 8086 y 8088

Aparecen en los meses de Junio de 1978-79 los procesadores 8086 y 8088. Implementan el pipe line o solapamiento de procesos. Representa una mejora en la cantidad de instrucciones que se procesan por segundo. El pipe line permite procesar simultáneamente diversas etapas de distintas instrucciones, completándose en cada etapa una parte de la ejecución de cada instrucción.

El 8088 se diseña con un bus de 8 bits en vez de 16 bits como el que tiene el 8086. Para resolver esta diferencia Intel divide cada procesador 8088 y 8086 en dos Sub-procesadores:

- Unidad de Ejecución (EU): realiza todas las operaciones.
- Unidad de Interfaz con el Bus (BIU): accede a datos e instrucciones del mundo exterior.

En ambos procesadores las Unidades de Ejecución son idénticas pero varía la Unidad de Interfaz con el bus. Así se consiguió un ahorro de esfuerzo para producir el 8088

Estos procesadores tienen 27 modos de direccionamiento para localizar un operando de una instrucción.

Las principales aportaciones son:

- Gestión de memoria
- Segmentación de 2 etapas: Buscar instrucción y ejecutar.
- Interrupciones sectorizadas multinivel.

La segunda generación: 80286

Aparecen a principios de febrero de 1982. Se caracterizan por poseer dos modos de funcionamiento completamente diferenciados:

- Modo Real: Se comporta igual que un 8086 pero con mayor velocidad, al ser conectado a la alimentación arranca en este modo.
- Modo Protegido: funciona con capacidad de proceso multitarea y memoria virtual. Este modo es propio del 286 donde todas las extensiones se ponen en marcha, por lo que pierde la compatibilidad con los procesadores anteriores. Cuando la CPU está en modo protegido, los programas de usuario tienen un acceso limitado al juego de instrucciones; solo el proceso supervisor está capacitado para realizar ciertas tareas. Así se evitan posibles conflictos entre los distintos programas de usuario, con lo que el fallo de un proceso no afecta al resto.

Las principales aportaciones son:

- Memoria Virtual hasta 1 GB y Memoria Física hasta 16 MB.
- Admitía multitarea e introdujo sistemas de protección.
- Cuatro niveles de privilegio.
- Aumenta la segmentación a cuatro capas.

Tercera generación: 80386

Es el primer procesador de 32 bits del mundo y ha llegado a ser un estándar en la industria.

Tiene tres modos posibles de funcionamiento:

- Modo Real: Compatible con el 8086.
- Modo Protegido: propio que le permite romper la barrera de los tradicionales segmentos.
- Modo Virtual 86: puede emular el funcionamiento simultáneo de varios 8086.

El sistema de paginación es transparente a la segmentación y permite el manejo de direcciones físicas. Cada segmento se divide en una o más páginas de cuatro kilobytes.

La unidad de segmentación provee cuatro niveles de protección para aislar y proteger aplicaciones y el sistema operativo

Para facilitar diseños de hardware de alto rendimiento, la interfaz con el bus (BIU) del 80386 ofrece pipelining de direcciones, tamaño dinámico del ancho del bus de datos (16 o 32 bits) y señales de habilitación de bytes por cada byte del bus de datos.

Cuarta generación: 80486

El 80486 es una versión mejorada del 80386 que además tiene integrada una cache de 8 Kbytes y un coprocesador matemático 80387, con lo que se consigue que casi la mitad de las instrucciones del 486 se ejecuten en un periodo de reloj en vez de los dos periodos que requiere el 386.

Los principales bloques que están en el interior de un procesador 486 y sus funciones son:

- **Los registros de direcciones (RDI) y de datos (RDA)**, pertenecientes a la unidad de interconexión con el bus (BIU) encargada de la comunicación con el exterior a través de las 32 líneas de datos y 32 líneas de direcciones del bus. Estos registros cumplen las mismas funciones que en la arquitectura von Neumann solo que ahora las instrucciones y los datos leídos en memoria pasan al caché interno de 8KB del procesador.
- **La unidad de caché de 8KB** guarda las instrucciones y datos que seguramente serán requeridos próximamente. A través de un bus de 128 líneas se pueden leer del caché 16 bytes (128/8) que pasan a un buffer de la unidad de pre-carga de instrucciones. Corresponden en promedio a unas 5 instrucciones a ejecutar que llegan juntas para entrar al pipe line.
- **La unidad de pre-carga** proporciona las direcciones de las próximas instrucciones a ejecutar y guarda las mismas en orden en dos buffers de 16 bytes, para que luego cada una sea decodificada.
- **La unidad de decodificación** realiza dos decodificaciones de cada instrucción.
- **La unidad de control** mediante líneas que salen de ella activa las operaciones que con cada pulso de reloj deben realizar los distintos bloques de la UCP (U de pre-carga, decodificadora, UAL, UPF, UC) conforme lo establecen micrócodigos de la ROM de control
- **La unidad de segmentación, paginación y protección de memoria o unidad de manejo de memoria (MMU)** se encarga de proporcionar las direcciones físicas de memoria que utiliza un programa y de la protección contra escrituras no permitidas en zonas reservadas de memoria.

Todas estas unidades participan del pipe line de instrucciones que en el 486 consta de 5 etapas:

1. **Pre-carga:** Consiste en la llegada de los códigos de las próximas instrucciones que entrarán al pipe-line a dos buffers de la unidad de pre-carga, para formar una cola.
2. **Primera decodificación:** A la unidad de decodificación llegan los primeros 3 bytes de cada instrucción, para separar su código de operación del número que hace referencia la dirección del dato.
3. **Segunda decodificación:** El código de operación es decodificado, lo que permite determinar la secuencia del micrócodigos contenida en la ROM de control. La UC generará

las señales de control que enviará por líneas que salen de ella, para que cada unidad que controla, ejecute una parte de la instrucción con cada pulso reloj.

4. **Ejecución** de cada instrucción en la unidad correspondiente.
5. **Almacenamiento de resultados:** en esta etapa final del pipe line, llega la primer instrucción completándose su ejecución. En caso de generar datos, estos se almacenan en los registros correspondientes. En este momento la instrucción 2 (I_2), ingresa en etapa de ejecución, I_3 entra a la segunda decodificación y los bytes de I_4 son sometidos a la primera.

Quinta generación: Pentium

Las ventajas que aporta son:

- Supersegmentación con 14 etapas. Técnicas de predicción de saltos condicionales para evitar introducir demasiadas burbujas.
- Arquitectura superescalar. Dos cauces de datos, en un ciclo se ejecutan más de una instrucción.
- Aumento de cache. Caché de primer nivel (L1) y cache de segundo nivel (L2). La caché L1 se divide en dos partes independientes para datos para instrucciones de 8 KB cada una, con lo que es posible acceder a un dato y una instrucción en paralelo.
- El chip se empaqueta en formato PGA (Pin Grid Array) de 273 pines.

Las 7 unidades funcionales que aportan características específicas e innovadoras al procesador consiguiendo altas prestaciones, compatibilidad y mantenimiento de la integridad de los datos son:

- Unidad de enteros Superescalar: consiste en dos unidades de enteros de 32 bits que operan en paralelo.
- Unidades de memoria caché: están subdivididas en dos memorias caché independientes, una para datos y otra para instrucciones.
- Unidad de interconexión con el bus: presenta un bus de datos de 64 bits con lo que se obtiene una velocidad de transferencia de 538 MB/s.
- Monitor de prestaciones: consta de una serie de controladores internos y unidades de rastreo para evitar que se pierda gran cantidad de tiempo en ciertas rutinas o secciones de código.
- Unidad de redundancia funcional: consiste en una serie de técnicas para asegurar la integridad de los datos.
- Unidad de predicción de bifurcaciones: consta de una caché específica encargada de hacer una predicción dinámica de los saltos condicionales.
- Unidad de coma flotante: ha sido mejorada respecto del 486 incorporando un cauce segmentado de instrucciones de 8 etapas.
- Vías de acceso múltiple: proporcionan una arquitectura superescalar que tiene la habilidad de ejecutar más de una instrucción por cada ciclo de reloj.

¿Qué tienen en común y cómo funciona el Pentium en relación al 486?

En el Pentium las instrucciones para enteros siguen un pipe line de 5 etapas para números enteros y de 8 etapas para flotantes con coma.

Diferencias con el 486

Para la etapa de pre-carga (Se supone que en un caché interno de 8KB se encuentran las próximas instrucciones a ejecutar) pasan en promedio diez instrucciones juntas hacia un buffer de la U de pre-carga que puede almacenar 32 bytes. O sea, en dicho caché se leen 32 bytes en un solo acceso. Los datos están en otro caché de 8KB.

Tener dos memorias caché separadas permite que mientras por un lado se accede a las próximas instrucciones a ejecutar en un caché, al mismo tiempo en el otro, se accede a datos, sin tener que esperar.

El bus de datos de 64 bits interno y externo del Pentium, posibilita que en cada acceso al caché externo, cada caché reciba el doble de datos o instrucciones que en el 486.

Los dos pipeline de 5 instrucciones del Pentium le permiten terminar de ejecutar dos instrucciones por pulso. A la primer decodificación entran dos instrucciones al mismo tiempo, lo que determinará si ambas se procesarán juntas (Una en cada pipe line) o si sólo seguirá una por el pipe line. Se identifica la porción de cada instrucción que permite formar la dirección del dato y cada código de operación.

Las denominadas instrucciones simples para enteros, luego de haber pasado por la pre-carga y las dos decodificaciones se ejecutan en uno, dos o tres pulsos de reloj, según sea su complejidad. No requieren acceder a microcódigos de la ROM de control: A partir de su código de operación se generan las señales de control que ordenarán a los circuitos de la UCP intervinientes, que operaciones realizarán.

Arquitectura del Pentium:

- 2 memorias caché: 1 para acceder a las próximas instrucciones a ejecutar y otra para acceder a los datos. Ambas operan simultáneamente para no esperar.
- Bus de datos externo e interno de 64 bits que llega a cada caché
- 2 pipe line capaces de procesar dos instrucciones de manera independiente.
- Dos unidades decodificadoras, para decodificar dos instrucciones a la vez (Una para cada pipe line)
- Dos unidades de segmentación generadora de direcciones de datos
- Dos UAL.

Anexo

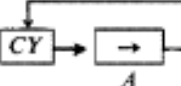
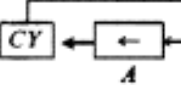
Unidad V: El microprocesador

Resumen de instrucciones

Instrucciones aritméticas

Descripción de la operación	Modo de direccionamiento	Nemotécnico	Cód. de oper.	Bytes	Formato de la instrucción	Simbólico	Señalizadores afectados
Suma A al dato	Inmediato	ADI	C6	2	Código de op. datos	$(A) \leftarrow (A) + (\text{byte } 2)$	Z, CY
Suma L a A	Registro	ADD L	85	1	Código de op.	$(A) \leftarrow (A) + (L)$	Z, CY
Suma H a A	Registro	ADD H	84	1	Código de op.	$(A) \leftarrow (A) + (H)$	Z, CY
Suma LOC (H y L) a A	Indirecto de registro	ADD M	86	1	Código de op.	$(A) \leftarrow (A) + ((H)(L))$	Z, CY
Resta el dato de A	Inmediato	SUI	D6	2	Código de op. datos	$(A) \leftarrow (A) - (\text{byte } 2)$	Z, CY
Resta L de A	Registro	SUB L	95	1	Código de op.	$(A) \leftarrow (A) - (L)$	Z, CY
Resta H de A	Registro	SUB H	94	1	Código de op.	$(A) \leftarrow (A) - (H)$	Z, CY
Resta LOC (H y L) de A	Indirecto de registro	SUB M	96	1	Código de op.	$(A) \leftarrow (A) - ((H)(L))$	Z, CY
Incrementa A	Registro	INC A	3C	1	Código de op.	$(A) \leftarrow (A) + 1$	Z
Incrementa HL	Registro	INX H	23	1	Código de op.	$(HL) \leftarrow (HL) + 1$	
Decrementa A	Registro	DCR A	3D	1	Código de op.	$(A) \leftarrow (A) - 1$	Z
Decrementa HL	Registro	DCX H	2B	1	Código de op.	$(HL) \leftarrow (HL) - 1$	
Compara A con el dato	Inmediato	CPI	FE	2	Código de op. datos	$(A) - (\text{byte } 2)$	$Z = 1$ si $(A) = (\text{byte } 2)$ $CY = 1$ si $(A) < (\text{byte } 2)$
Compara A con L	Registro	CMP L	BD	1	Código de op.	$(A) - (L)$	$Z = 1$ si $(A) = (L)$ $CY = 1$ si $(A) < (L)$
Compara A con H	Registro	CMP H	BC	1	Código de op.	$(A) - (H)$	$Z = 1$ si $(A) = (H)$ $CY = 1$ si $(A) < (H)$
Compara A con LOC (H y L)	Indirecto de registro	CMP M	BE	1	Código de op.	$(A) - ((H)(L))$	$Z = 1$ si $(A) = ((H)(L))$ $CY = 1$ si $(A) < ((H)(L))$

Instrucciones lógicas

Descripción de la operación	Modo de direccionamiento	Nemotécnico	Cód. de oper.	Bytes	Formato de la instrucción	Simbólico	Señalizadores afectados
AND de A con dato	Inmediato	ANI	E6	2	Código de op. dato	$(A) \leftarrow (A) \cdot (\text{byte } 2)$	Z CY se borra
AND de A con L	Registro	ANA L	A5	1	Código de op.	$(A) \leftarrow (A) \cdot (L)$	Z CY se borra
AND de A con H	Registro	ANA H	A4	1	Código de op.	$(A) \leftarrow (A) \cdot (H)$	Z CY se borra
AND de A con LOC ($H \& L$)	Indirecto de registro	ANA M	A6	1	Código de op.	$(A) \leftarrow (A) \cdot ((H)(L))$	Z CY se borra
OR de A con dato	Inmediato	ORI	F6	2	Código de op. dato	$(A) \leftarrow (A) + (\text{byte } 2)$	Z CY se borra
OR de A con L	Registro	ORA L	B5	1	Código de op.	$(A) \leftarrow (A) + (L)$	Z CY se borra
OR de A con H	Registro	ORA H	B4	1	Código de op.	$(A) \leftarrow (A) + (H)$	Z CY se borra
OR de A con LOC ($H \& L$)	Indirecto de registro	ORA M	B6	1	Código de op.	$(A) \leftarrow (A) + ((H)(L))$	Z CY se borra
OR exclusivo de A con dato	Inmediato	XRI	EE	2	Código de op. dato	$(A) \leftarrow (A) \oplus (\text{byte } 2)$	Z CY se borra
OR exclusivo de A con A	Registro	XRA A	AF	1	Código de op.	Borra el acumulador $(A) \leftarrow (A) \oplus (A)$	$Z = 1$ CY se borra
OR exclusivo de A con L	Registro	XRA L	AD	1	Código de op.	$(A) \leftarrow (A) \oplus (L)$	Z CY se borra
OR exclusivo de A con H	Registro	XRA H	AC	1	Código de op.	$(A) \leftarrow (A) \oplus (H)$	Z CY se borra
OR exclusivo de A con LOC ($H \& L$)	Indirecto de registro	XRA M	AE	1	Código de op.	$(A) \leftarrow (A) \oplus ((H)(L))$	Z CY se borra
Complementar A (complemento a 1)	Inherente	CMA	2F	1	Código de op.	$(A) \leftarrow (\bar{A})$	
Desplazamiento circular a la derecha a través del arrastre	Inherente	RAR	1F	1	Código de op.		CY
Desplazamiento circular a la izquierda a través del arrastre	Inherente	RAL	17	1	Código de op.		CY

() contenido

(()) = direccionamiento indirecto de registro

$\cdot$ = AND

$+$ = OR

$\oplus$ = XOR

Instrucciones de transferencia de datos

Descripción de la operación	Modo de direccionamiento	Nemotécnico	Cód. de oper.	Bytes	Formato de la instrucción	Simbólico	Señalizadores afectados
Transfiere <i>L</i> a <i>A</i>	Registro	MOV <i>A,L</i>	7D	1	Código de op.	$(A) \leftarrow (L)$	
Transfiere <i>H</i> a <i>A</i>	Registro	MOV <i>A,H</i>	7C	1	Código de op.	$(A) \leftarrow (H)$	
Transfiere <i>A</i> a <i>L</i>	Registro	MOV <i>LA</i>	6F	1	Código de op.	$(L) \leftarrow (A)$	
Transfiere <i>A</i> a <i>H</i>	Registro	MOV <i>HA</i>	67	1	Código de op.	$(H) \leftarrow (A)$	
Transfiere <i>HL</i> a <i>PC</i>	Registro	PCHL	E9	1	Código de op.	$(PC) \leftarrow (HL)$	
Transfiere <i>HL</i> a <i>SP</i>	Registro	SPHL	F9	1	Código de op.	$(SP) \leftarrow (HL)$	
Carga <i>A</i> con dato	Inmediato	MVI <i>A</i>	3E	2	Código de op. datos	$(A) \leftarrow (\text{byte } 2)$	
Carga <i>L</i> con dato	Inmediato	MVI <i>L</i>	2E	2	Código de op. datos	$(L) \leftarrow (\text{byte } 2)$	
Carga <i>H</i> con dato	Inmediato	MVI <i>H</i>	26	2	Código de op. datos	$(H) \leftarrow (\text{byte } 2)$	
Carga LOC (<i>H</i> y <i>L</i>) a <i>A</i>	Indirecto de registro	MOV <i>A,M</i>	7E	1	Código de op.	$(A) \leftarrow ((H)(L))$	
Carga <i>HL</i> con dato	Inmediato	LXI <i>H</i>	21	3	Código de op. dato de orden inferior dato de orden superior	$(L) \leftarrow (\text{byte } 2)$ $(H) \leftarrow (\text{byte } 3)$	

Instrucciones de transferencia de datos

Descripción de la operación	Modo de direccionamiento	Nemotécnico	Cód. de oper.	Bytes	Formato de la instrucción	Simbólico	Señalizadores afectados
Carga <i>SP</i> con dato	Inmediato	LXI <i>SP</i>	31	3	Código de op. dato de orden inferior dato de orden superior	$(SP) \leftarrow (\text{byte } 2 + 3)$	
Carga <i>HL</i> de LOC <i>aa</i>	Directo	LHLD	2A	3	Código de op. direc. de orden inferior direc. de orden superior	$(L) \leftarrow ((\text{byte } 2 + 3))$ $(H) \leftarrow ((\text{byte } 2 + 3) + 1)$	
Carga <i>A</i> de LOC <i>aa</i>	Directo	LDA	3A	3	Código de op. direc. de orden inferior direc. de orden superior	$(A) \leftarrow ((\text{byte } 2 + 3))$	
Almacena <i>A</i> en LOC <i>aa</i>	Directo	STA	32	3	Código de op. direc. de orden inferior direc. de orden superior	$(\text{Dirección}) \leftarrow (A)$	
Almacena <i>HL</i> en LOC <i>aa</i>	Directo	SHLD	22	3	Código de op. direc. de orden inferior direc. de orden superior	$(\text{Dirección}) \leftarrow (L)$ $(\text{Dirección} + 1) \leftarrow (H)$	
Almacena <i>A</i> en LOC (<i>H</i> y <i>L</i>)	Indirecto de registro	MOV <i>MA</i>	77	1	Código de op.	$((H)(L)) \leftarrow (A)$	
Almacena <i>H</i> en LOC (<i>H</i> y <i>L</i>)	Indirecto de registro	MOV <i>ML</i>	75	1	Código de op.	$((H)(L)) \leftarrow (L)$	
Almacena <i>H</i> en LOC (<i>H</i> y <i>L</i>)	Indirecto de registro	MOV <i>MH</i>	74	1	Código de op.	$((H)(L)) \leftarrow (H)$	
Entrada de <i>A</i> del puerto en LOC <i>a</i>	Directo	IN	D8	2	Código de op. direc. del puerto	$(A) \leftarrow (\text{Dirección del puerto})$	
Salida de <i>A</i> al puerto en LOC <i>a</i>	Directo	OUT	D3	2	Código de op. direc. del puerto	$(\text{Dirección del puerto}) \leftarrow (A)$	
Coloca señalizador de arrastre	Inherente	STC	37	1	Código de op.	$(CY) \leftarrow 1$	<i>CY</i> se pone a 1

(()) = contenido de

(()) = direccionamiento indirecto de registro

PC = contador de programa

SP = puntero de pila

Instrucciones de bifurcación

Descripción de la operación	Modo de direccionamiento	Nemotécnico	Cód. de oper.	Bytes	Formato de la instrucción	Simbólico	Señalizadores afectados
Salta a LOC <i>aa</i>	Inmediato	JMP	C3	3	Código de op. Direc. de orden inferior Direc. de orden superior	$(PC) \leftarrow (\text{Dirección})$	
Salta a LOC <i>aa</i> si es 0	Inmediato	JZ	CA	3	Código de op. Direc. de orden inferior Direc. de orden superior	Si el señalizador de 0 = 1, entonces $(PC) \leftarrow (\text{dirección})$	
Salta a LOC <i>aa</i> si no es 0	Inmediato	JNZ	C2	3	Código de op. Direc. de orden inferior Direc. de orden superior	Si el señalizador de 0 = 0, entonces $(PC) \leftarrow (\text{dirección})$	
Salta a LOC <i>aa</i> si se coloca el señalizador de arrastre	Inmediato	JC	DA	3	Código de op. Direc. de orden inferior Direc. de orden superior	Si el señalizador de arrastre = 1, entonces $(PC) \leftarrow (\text{dirección})$	
Salta a LOC <i>aa</i> si no se coloca el señalizador de arrastre	Inmediato	JNC	D2	3	Código de op. Direc. de orden inferior Direc. de orden superior	Si el señalizador de arrastre = 0, entonces $(PC) \leftarrow (\text{dirección})$	

() = contenido de
PC = contador de programa

Instrucciones de llamada y vuelta de subrutina

Descripción de la operación	Modo de direccionamiento	Nemotécnico	Cód. de oper.	Bytes	Formato de la instrucción	Simbólico	Señalizadores afectados
Llama a la subrutina en LOC <i>aa</i>	Inmediato/indirecto de registro	CALL	CD	3	Código de op. Direc. de orden inferior Direc. de orden superior	$((SP) - 1) \leftarrow (PCH)$ $((SP) - 2) \leftarrow (PCL)$ $(SP) \leftarrow (SP) - 2$ $(PC) \leftarrow (\text{dirección})$	
Vuelta de subrutina	Indirecto de registro	RET	C9	1	Código de op.	$(PCL) \leftarrow ((SP))$ $(PCH) \leftarrow ((SP) + 1)$ $(SP) \leftarrow (SP) + 2$	

() = contenido de
(()) = direccionamiento indirecto de registro
PC = contador de programa
SP = puntero de pila

Bibliografía:

Arquitectura de computadoras - Patricia Quiroga - Alfaomega - 2010
 La PC por dentro 4ª edición - Mario Guinzburg - 2006
 Fundamento de los microprocesadores - Roger L. Tokheim - Mc Graw Hill - Segunda edición